Constructive uniform non-idempotent intersection typing of simply typed lambda terms

Andrej Dudenhefner, Daniele Pautasso

doi:10.4230/LIPIcs.FSCD.2024.8; Unlicense (author Coq source); CC BY4.0 (article)

Editorial note. Only the original constructive uniform typing theorem nitlc_type_inference is counted. STLC strong normalization, reverse collapse and individual expansion lemmas are supporting material, not new items.

Editorial note. Fixed official author software revision31a02d2 is Unlicense, preserved with its LICENSE/README/build files. All seven original formal source files and complete Proof/Qed commands are printed as editable text and retained separately. The related official article is CC BY4.0 and its complete PDF is attached.

Editorial note. The proof representation is original author Coq code. It has not been executed or independently certified here. The two Greek symbols appearing in source comments are rendered through equivalent TeX glyphs only; source files are unchanged.

Editorial scope. Complete author material collection for one structural theorem; author retains responsibility for the mathematics.

Allowed standard prerequisites (editorial).

- Coq8.19 standard library and SSReflect/ssrfun. Original README, Require/From imports and project build files retained; no compilation performed.
- De Bruijn lambda syntax, simple typing, uniform intersection typing and collapse relation. Complete original stlc.v/nitlc.v definitions and all local dependency files are included as editable formal text, not external proof pointers.

Original author formal source: facts.v

```
Require Import Relations List Lia PeanoNat Permutation Wellfounded.
Import ListNotations.
Require Import ssreflect ssrfun.

Arguments nth_error_split {A l n a}.
Arguments Permutation_nil_cons {A l x}.
Arguments in_split {A x}.

(* point-wise list concatenation *)
Fixpoint merge {X : Type} (Gamma : list (list X)) (Delta : list (list X)) : list (list X) :=
  match Gamma with
  | nil => Delta
  | cons u Gamma =>
    match Delta with
    | nil => cons u Gamma
    | cons v Delta => cons (u ++ v) (merge Gamma Delta)
    end
  end.

Lemma Forall2E {X Y : Type} {P : X -> Y -> Prop} l1 l2 : Forall2 P l1 l2 ->
  match l1 with
  | nil => l2 = nil
  | cons x l1 =>
      match l2 with
      | nil => False
      | cons y l2 => P x y /\ Forall2 P l1 l2
      end
  end.
Proof.
  by case.
Qed.

Lemma Forall2_trans {X Y Z : Type} (P : X -> Y -> Prop) (Q : Y -> Z -> Prop) (R : X -> Z -> Prop) l1 l2 l3 :
  (forall x y z, P x y -> Q y z -> R x z) -> Forall2 P l1 l2 -> Forall2 Q l2 l3 -> Forall2 R l1 l3.
Proof.
  move=> HPQR H. elim: H l3.
  - move=> ? /Forall2E ->. by constructor.
  - move=> > ??? [|? l3] /Forall2E; [done|].
    move=> [? ?]. constructor; by eauto.
Qed.

Lemma Forall2_map_l {X Y Z : Type} (P : X -> Y -> Prop) (f : Z -> X) l1 l2 :
  Forall2 (fun x y => P (f x) y) l1 l2 -> Forall2 P (map f l1) l2.
Proof.
  elim.
  - constructor.
  - move=> * /=. by constructor.
Qed.

Lemma Forall2_map_r {X Y Z : Type} (P : X -> Y -> Prop) (f : Z -> Y) l1 l2 :
  Forall2 (fun x y => P x (f y)) l1 l2 -> Forall2 P l1 (map f l2).
Proof.
  elim.
  - constructor.
  - move=> * /=. by constructor.
Qed.

Lemma Forall2_eq {X : Type} (P : X -> X -> Prop) l :
  Forall (fun x => P x x) l -> Forall2 P l l.
Proof. elim; by constructor. Qed.

Lemma merge_nil_r {X : Type} (Gamma : list (list X)) : merge Gamma nil = Gamma.
Proof.
  by case: Gamma.
Qed.

Lemma cons_app {X : Type} (x : X) x1s x2s : x :: x1s ++ x2s = [x] ++ x1s ++ x2s.
Proof. done. Qed.

Lemma map_neq_nil {X Y : Type} {f : X -> Y} l : map f l <> [] -> l <> [].
Proof.
  by case: l.
Qed.

Lemma nth_merge {X : Type} (Gamma1 Gamma2 : list (list X)) x :
  nth x (merge Gamma1 Gamma2) nil = nth x Gamma1 nil ++ nth x Gamma2 nil.
Proof.
  elim: x Gamma1 Gamma2.
  - by move=> [|??] [|??]; rewrite /= ?app_nil_r.
  - by move=> x IH [|??] [|??]; rewrite /= ?app_nil_r.
Qed.

Lemma In_nth_merge_l {X : Type} (Gamma2 Gamma1 : list (list X)) A x :
  In A (nth x Gamma1 nil) -> In A (nth x (merge Gamma1 Gamma2) nil).
```

```
Proof.
  move=> ?. rewrite nth_merge. apply /in_app_iff. by left.
Qed.

Lemma In_nth_merge_r {X : Type} (Gamma1 Gamma2 : list (list X)) A x :
  In A (nth x Gamma2 nil) -> In A (nth x (merge Gamma1 Gamma2) nil).
Proof.
  move=> ?. rewrite nth_merge. apply /in_app_iff. by right.
Qed.

Lemma nth_map_seq_lt {X : Type} {f : nat -> X} {i n d} : i < n -> nth i (map f (seq 0 n)) d = f i.
Proof.
  suff: (forall k, i < n -> nth i (map f (seq k n)) d = f (k + i)).
  { move=> /(_ 0). by apply. }
  elim: n i.
  - lia.
  - move=> n IH [|i] k ? /=.
    + by rewrite Nat.add_0_r.
    + rewrite IH; [lia|].
      by rewrite Nat.add_succ_r.
Qed.

Lemma nth_map_seq_ge {X : Type} {f : nat -> X} {i n d} : i >= n -> nth i (map f (seq 0 n)) d = d.
Proof.
  suff: (forall k, i >= n -> nth i (map f (seq k n)) d = d).
  { move=> /(_ 0). by apply. }
  elim: n i.
  - by case.
  - move=> n IH [|i] k ? /=.
    + lia.
    + by rewrite IH; [lia|].
Qed.

Fixpoint replace {X : Type} (xs : list X) (n : nat) (l : list (list X)) :=
  match n with
  | 0 => match l with
         | [] => [xs]
         | ys :: l => xs :: l
         end
  | S n =>
      match l with
      | [] => [] :: replace xs n l
      | ys :: l => ys :: replace xs n l
      end
  end.

Lemma nth_replace {X : Type} {xs : list X} {i l} : nth i (replace xs i l) [] = xs.
Proof.
  elim: i l.
  - by case.
  - move=> i IH [|??]; by apply: IH.
Qed.

Lemma nth_replace_neq {X : Type} {xs : list X} {i j l} : i <> j -> nth i (replace xs j l) [] = nth i l [].
Proof.
  elim: i j l.
  - by move=> [|?] [|??] /=.
  - move=> i IH [|j] [|??] /= ?.
    + by case: (i).
    + done.
    + rewrite IH; [lia|]. by case: (i).
    + apply: IH. lia.
Qed.

Lemma In_nth {X : Type} {n l} {x : X} : In x (nth n l []) ->
  exists l1 ys1 ys2 l2, l = l1 ++ (ys1 ++ x :: ys2) :: l2 /\ length l1 = n.
Proof.
  elim: n l.
  - move=> [|y l] /=; first done.
    move=> /in_split [ys1] [ys2] ->.
    by exists nil, ys1, ys2, l.
  - move=> n IH [|y l] /=; first done.
    move=> /IH [l1] [ys1] [ys2] [l2] [->] <-.
    by exists (y :: l1), ys1, ys2, l2.
Qed.

Lemma nth_middle_neq {X : Type} {n l1 l2 d} (x y : X) :
  n <> length l1 ->
  nth n (l1 ++ x :: l2) d = nth n (l1 ++ y :: l2) d.
Proof.
  elim: n l1.
  * by case.
  * move=> n IH [|??] /= ?; first done.
    apply: IH. lia.
Qed.

Fixpoint insert {X : Type} (x : X) n l :=
```

```
  match n with
  | 0 =>
    match l with
    | [] => [[x]]
    | xs :: l => (x :: xs) :: l
    end
  | S n =>
    match l with
    | [] => [] :: insert x n l
    | xs :: l => xs :: insert x n l
    end
  end.

Lemma nth_insert_neq {X : Type} {x : X} n1 n2 l : n1 <> n2 -> nth n1 (insert x n2 l) [] = nth n1 l [].
Proof.
  elim: n2 n1 l.
  - by move=> [|[|n1]] [|??] /=.
  - move=> n2 IH [|n1] [|??] /= ?; [done|done|..].
    * rewrite IH; [lia|]. by case: (n1).
    * apply: IH. lia.
Qed.

Lemma nth_insert {X : Type} {x : X} n l :
  nth n (insert x n l) [] = x :: nth n l [].
Proof.
  elim: n l.
  - by case.
  - move=> n IH [|??] /=; rewrite IH; by case: (n).
Qed.

Lemma nth_error_combine {X Y : Type} {n} {l1 : list X} {l2 : list Y} {x y} : nth_error (combine l1 l2) n =
    Some (x, y) <->
  (nth_error l1 n = Some x /\ nth_error l2 n = Some y).
Proof.
  split.
  - elim: l1 l2 n.
    + by move=> [|??] [|?] /=.
    + move=> ?? IH [|??] [|n] /= ?; [split; congruence..|].
      by apply: IH.
  - elim: l1 l2 n.
    + by move=> [|??] [|?] [] /=.
    + move=> ?? IH [|??] [|?] /= []; [congruence..|].
      move=> *. by apply: IH.
Qed.

Lemma map_nil {X Y : Type} (f : X -> Y) : map f [] = [].
Proof. done. Qed.

Lemma list_sum_cons n ns : list_sum (n :: ns) = n + list_sum ns.
Proof. done. Qed.

Lemma nth_error_length_neq {X Y : Type} (l1 : list X) (l2 : list Y) n :
  nth_error l1 n <> None ->
  nth_error l2 n = None ->
  length l1 <> length l2.
Proof.
  move=> /nth_error_Some ? /nth_error_None ?. lia.
Qed.

Lemma Forall2_concat {X Y : Type} (P : X -> Y -> Prop) (l1s : list (list X)) (l2s : list (list Y)) :
  Forall2 (Forall2 P) l1s l2s -> Forall2 P (concat l1s) (concat l2s).
Proof.
  elim; first done.
  move=> * /=. by apply: Forall2_app.
Qed.

Lemma In_In_combine_l {X Y : Type} {x : X} l1 l2 : In x l1 -> length l1 = length l2 -> exists (y : Y), In (x
    , y) (combine l1 l2).
Proof.
  elim: l1 l2; first done.
  move=> ?? IH [|??] /=; first done.
  move=> [?|/IH].
  - subst=> ?. eexists. by left.
  - move=> H [/H] [??].
    eexists. right. by eassumption.
Qed.

Lemma In_In_combine_r {X Y : Type} {y : Y} l1 l2 : In y l2 -> length l1 = length l2 -> exists (x : X), In (x
    , y) (combine l1 l2).
Proof.
  elim: l2 l1; first done.
  move=> ?? IH [|??] /=; first done.
  move=> [?|/IH].
  - subst=> ?. eexists. by left.
  - move=> H [/H] [??].
    eexists. right. by eassumption.
Qed.
```

```
Lemma Forall2_Forall {X Y : Type} (P : X -> Y -> Prop) l1 l2 :
  (Forall (fun xy => P (fst xy) (snd xy)) (combine l1 l2) /\ length l1 = length l2) <-> Forall2 P l1 l2.
Proof.
  split.
  - elim: l1 l2.
    + by move=> [|??] [].
    + move=> ?? IH [|??] [] /=; first done.
      move=> /Forall_cons_iff [??] [?].
      constructor; first done.
      by apply: IH.
  - elim; first done.
    move=> > ?? /= [? ->]. by split; [constructor|].
Qed.

Lemma Forall2_exists_exists_Forall2 {X Y Z : Type} (P : X -> Y -> Z -> Prop) l1 l2 :
  Forall2 (fun x y => exists z, P x y z) l1 l2 ->
  exists zs, Forall2 (fun z xy => P (fst xy) (snd xy) z) zs (combine l1 l2).
Proof.
  elim.
  - by exists nil.
  - move=> > [z ?] ? [zs ?]. exists (z :: zs). by constructor.
Qed.

Lemma Permutation_Forall {A : Type} (P : A -> Prop) (l1 l2 : list A) : Permutation l1 l2 -> Forall P l1 ->
     Forall P l2.
Proof.
  elim.
  - done.
  - move=> > ?? /Forall_cons_iff [??]. by constructor; auto.
  - move=> > /Forall_cons_iff [? /Forall_cons_iff [??]]. by do ? constructor.
  - by auto.
Qed.

Lemma nth_skipn {A : Type} n1 n2 (a : A) l : nth n1 (skipn n2 l) a = nth (n2 + n1) l a.
Proof.
  elim: n2 l; first done.
  move=> n2 IH [|??] /=.
  - by case: (n1).
  - by apply: IH.
Qed.

Lemma concat_cons {A : Type} (l1 : list A) ls : concat (l1 :: ls) = l1 ++ concat ls.
Proof. done. Qed.

Lemma Permutation_eq {X : Type} {l1 l2 : list X} : l1 = l2 -> Permutation l1 l2.
Proof. by move=> ->. Qed.

Lemma Forall2_permutation_app {X Y : Type} (P : X -> Y -> Prop) l l' l1 l2 : Forall2 P l l' -> Permutation l
      (l1 ++ l2) ->
  exists l'1 l'2, Permutation l' (l'1 ++ l'2) /\ Forall2 P l1 l'1 /\ Forall2 P l2 l'2.
Proof.
  move=> H. elim: H l1 l2.
  { move=> > /Permutation_nil /app_eq_nil [-> ->]. by exists nil, nil. }
  move=> x y {}l {}l' Hxy ? IH l1 l2.
  have : In x (x :: l) by left.
  move=> /(@Permutation_in X) + /[dup] => /[apply] /in_app_iff [].
  - move=> /in_split [l3] [l4] ?. subst.
    rewrite -app_assoc /=.
    move=> /(@Permutation_app_inv X nil).
    rewrite app_assoc=> /IH [l'3] [l'4] [?].
    move=> [/Forall2_app_inv_l] [l'31] [l'32] [?] [?] *. subst.
    exists (l'31 ++ y :: l'32), l'4. split; [|split].
    + rewrite -app_assoc /=.
      apply: (Permutation_app_middle [_] nil).
      by rewrite !app_assoc.
    + apply: Forall2_app; first done.
      by constructor.
    + done.
  - move=> /in_split [l3] [l4] ?. subst.
    rewrite app_assoc /=.
    move=> /(@Permutation_app_inv X nil).
    rewrite -app_assoc=> /IH [l'3] [l'4] [?] [?].
    move=> /Forall2_app_inv_l [l'41] [l'42] [?] [?] *. subst.
    exists l'3, (l'41 ++ y :: l'42). split; [|split].
    + rewrite app_assoc /=.
      apply: (Permutation_app_middle [_] nil).
      by rewrite -!app_assoc.
    + done.
    + apply: Forall2_app; first done.
      by constructor.
Qed.

Lemma wf_clos_trans_inverse_rel {A B : Type} (Q : relation B) (R : relation A) (F : B -> A -> Prop) :
  well_founded R ->
  (forall b, exists a, F b a) ->
  (forall b1 b2 a1, Q b2 b1 -> F b1 a1 -> exists a2, F b2 a2 /\ clos_trans _ R a2 a1) ->
```

```
  well_founded Q.
Proof.
  intros HR Hintro Hstep b.
  destruct (Hintro b) as [a H].
  revert b H.
  induction (wf_clos_trans _ _ HR a) as [a _ IH].
  intros b Hba.
  constructor.
  intros b' Hb'b.
  destruct (Hstep _ _ _ Hb'b Hba) as [a' [??]].
  eapply IH; eassumption.
Qed.

(* well-founded list extension of a well-founded relation *)
Section ListLt.

Context {X : Type} (ltX : X -> X -> Prop) (ltX_wf : forall x, Acc ltX x).

Inductive list_lt : list X -> list X -> Prop :=
  | list_lt_intro x (ys : list X) l1 l2 :
      Forall (fun y => ltX y x) ys ->
      list_lt (l1 ++ ys ++ l2) (l1 ++ x :: l2).

Lemma list_lt_nil_r l : not (list_lt l nil).
Proof.
  move E: nil => l' H.
  by case: H E=> ?? [|??].
Qed.

Lemma list_lt_wf_app l1 l2 : Acc list_lt l1 -> Acc list_lt l2 -> Acc list_lt (l1 ++ l2).
Proof.
  move=> H. elim: H l2=> {}l1 _ IH1 l2.
  elim=> {}l2 Hl2 IH2. move E: (l1 ++ l2)=> l.
  constructor=> l' H. case: H E=> x ys l3 l4 ?.
  move=> /app_eq_app=> - [l''] [].
  - move: l''=> [|??] /=.
    + rewrite app_nil_r=> - [??]. subst.
      apply: IH2. by apply: (list_lt_intro x ys nil).
    + move=> [? [??]]. subst. rewrite !app_assoc.
      apply: IH1.
      * rewrite -!app_assoc. by constructor.
      * constructor=> ?. by apply: Hl2.
  - move=> [??]. subst. rewrite -!app_assoc.
    apply: IH2. by constructor.
Qed.

Lemma Acc_list_lt : forall l, Forall (Acc ltX) l -> Acc list_lt l.
Proof.
  elim.
  - by constructor=> ? /list_lt_nil_r.
  - move=> x l Hl.
    move=> /Forall_cons_iff [Hx /Hl {}Hl].
    apply: (list_lt_wf_app [_]); last done.
    elim: Hx {l Hl}=> {}x _ IH.
    constructor=> l'. move E: ([x])=> l H.
    case: H E=> ?? [|?[|?[|??]]] [|??]; [|done..].
    move=> + [?] /=. subst. elim.
    + by constructor=> ? /list_lt_nil_r.
    + move=> *. apply: (list_lt_wf_app [_]); last done.
      by apply: IH.
Qed.

Lemma list_lt_wf : well_founded list_lt.
Proof.
  move=> l. apply: Acc_list_lt.
  apply /Forall_forall=> *. by apply: ltX_wf.
Qed.

Lemma clos_trans_list_lt_neq_nil l : l <> [] -> clos_trans (list X) list_lt [] l.
Proof.
  elim: l; first done.
  move=> x [|y l] IH _.
  - apply: t_step. by apply: (list_lt_intro x [] nil nil).
  - apply: t_trans.
    + by apply: IH.
    + apply: t_step.
      by apply: (list_lt_intro x [] nil (_ :: _)).
Qed.

Lemma list_lt_app_l l1 l2 l : list_lt l1 l2 -> list_lt (l1 ++ l) (l2 ++ l).
Proof.
  move=> [] *. rewrite -!app_assoc. by constructor.
Qed.

Lemma list_lt_app_r l l1 l2 : list_lt l1 l2 -> list_lt (l ++ l1) (l ++ l2).
Proof.
  move=> [] ?? l' *. rewrite !(app_assoc l l'). by constructor.
```

```
Qed.

Lemma clos_trans_list_lt_app_l l1 l2 l : clos_trans (list X) list_lt l1 l2 -> clos_trans (list X) list_lt (
     l1 ++ l) (l2 ++ l).
Proof.
  elim.
  - move=> *. apply: t_step. by apply: list_lt_app_l.
  - move=> *. apply: t_trans; by eassumption.
Qed.

Lemma clos_trans_list_lt_app_r l l1 l2 : clos_trans (list X) list_lt l1 l2 -> clos_trans (list X) list_lt (l
      ++ l1) (l ++ l2).
Proof.
  elim.
  - move=> *. apply: t_step. by apply: list_lt_app_r.
  - move=> *. apply: t_trans; by eassumption.
Qed.

Lemma clos_refl_trans_list_lt_app_l l1 l2 l : clos_refl_trans (list X) list_lt l1 l2 -> clos_refl_trans (
     list X) list_lt (l1 ++ l) (l2 ++ l).
Proof.
  elim.
  - move=> *. apply: rt_step. by apply: list_lt_app_l.
  - by constructor.
  - move=> *. apply: rt_trans; by eassumption.
Qed.

Lemma clos_refl_trans_list_lt_app_r l l1 l2 : clos_refl_trans (list X) list_lt l1 l2 -> clos_refl_trans (
     list X) list_lt (l ++ l1) (l ++ l2).
Proof.
  elim.
  - move=> *. apply: rt_step. by apply: list_lt_app_r.
  - by constructor.
  - move=> *. apply: rt_trans; by eassumption.
Qed.

End ListLt.
```

Original author formal source: stlc.v

```
(*
  definitions for the simply typed lambda-calculus
  "stlc Gamma M s" means that in the type environment Gamma the term M is assigned the type s
*)

Require Import List Relations.

(* function composition *)
Definition funcomp {X Y Z} (g : Y -> Z) (f : X -> Y) :=
  fun x => g (f x).

Arguments funcomp {X Y Z} (g f) /.

(* stream cons *)
Definition scons {X: Type} (x : X) (xi : nat -> X) :=
  fun n => match n with | 0 => x | S n => xi n end.

(* simple types *)
Inductive sty : Type :=
  | satom (x : nat)   (* type variable *)
  | sarr (s t : sty). (* function type *)

(* type-annotated lambda-terms *)
Inductive tm : Type :=
  | var (n : nat)           (* term variable *)
  | app (M N : tm)          (* application *)
  | lam (t : sty) (M : tm). (* type-annotated abstraction *)

(* parallel term renaming *)
Fixpoint ren (xi : nat -> nat) (t : tm) : tm  :=
  match t with
  | var x => var (xi x)
  | app M N => app (ren xi M) (ren xi N)
  | lam t M => lam t (ren (scons 0 (funcomp S xi)) M)
  end.

(* parallel term substitution *)
Fixpoint subst (sigma: nat -> tm) (s: tm) : tm :=
  match s with
  | var n => sigma n
  | app M N => app (subst sigma M) (subst sigma N)
  | lam t M => lam t (subst (scons (var 0) (funcomp (ren S) sigma)) M)
  end.

Fixpoint allfv (P : nat -> Prop) (M: tm) : Prop :=
  match M with
  | var n => P n
  | app M N => allfv P M /\ allfv P N
  | lam _ M =>  allfv (scons True P) M
  end.

(* simply typed lambda-calculus *)
(* stlc Gamma M s means that in the type environment Gamma the term M is assigned the type s *)
Inductive stlc (Gamma : list sty) : tm -> sty -> Prop :=
  | stlc_var x t : nth_error Gamma x = Some t -> stlc Gamma (var x) t
  | stlc_app M N s t : stlc Gamma M (sarr s t) -> stlc Gamma N s -> stlc Gamma (app M N) t
  | stlc_lam M s t : stlc (cons s Gamma) M t -> stlc Gamma (lam (sarr s t) M) (sarr s t).

(* normal form property *)
Inductive nf : tm -> Prop :=
  | nf_lam t M : nf M -> nf (lam t M)
  | nf_neutral M : neutral M -> nf M
with neutral : tm -> Prop :=
  | neutral_var x : neutral (var x)
  | neutral_app M N : neutral M -> nf N -> neutral (app M N).

(* head form property: x M1 .. Mn *)
Inductive hf : tm -> Prop :=
  | hf_var x : hf (var x)
  | hf_app M N : hf M -> hf (app M N).

Notation is_nonzero := (fun n : nat => match n with 0 => False | S _ => True end).

(* has_var_zero M means that ((lam M) N) is a lambda-I redex *)
Notation has_var_zero M := (not (allfv is_nonzero M)).

(* traditional I reduction *)
Inductive stepI : tm -> tm -> Prop :=
  | stepIRed t M N   : has_var_zero M -> stepI (app (lam t M) N) (subst (scons N var) M)
  | stepILam t M M'  : stepI M M' -> stepI (lam t M) (lam t M')
  | stepIAppL M M' N : stepI M M' -> stepI (app M N) (app M' N)
  | stepIAppR M N N' : stepI N N' -> stepI (app M N) (app M N').

(* specific K reduction *)
```

```
Inductive stepK : tm -> tm -> Prop :=
  (* lambda-K redex contraction *)
  | stepKRed t M N          : nf N -> stepK (app (lam t (ren S M)) N) M
  | stepKLam t M M'         : stepK M M' -> stepK (lam t M) (lam t M')
  | stepKNAppR M N N'       : hf M -> stepK N N' -> stepK (app M N) (app M N')
  | stepKLAppR t M N N'     : stepK N N' -> stepK (app (lam t (ren S M)) N) (app (lam t (ren S M)) N')
  | stepKAAppL M1 M2 M' N   : stepK (app M1 M2) M' -> stepK (app (app M1 M2) N) (app M' N).

Definition swap (x : nat) :=
  match x with
  | 0 => 1
  | 1 => 0
  | _ => x
  end.

(* gamma reduction [1] *)
(* [1] Kfoury, A. J., and J. B. Wells.
       New notions of reduction and non-semantic proofs of β-strong normalization in typed λ-calculi.
       Boston University Computer Science Department, 1994. *)
Inductive stepG : tm -> tm -> Prop :=
  | stepGRed s1 s2 t M N :
      stepG
        (app (lam (sarr s1 (sarr s2 t)) (lam (sarr s2 t) M)) N)
        (lam (sarr s2 t) (app (lam (sarr s1 t) (ren swap M)) (ren S N)))
  | stepGLam t M M'      : stepG M M' -> stepG (lam t M) (lam t M')
  | stepGAppL M M' N   : stepG M M' -> stepG (app M N) (app M' N)
  | stepGAppR M N N'   : stepG N N' -> stepG (app M N) (app M N').

(* union of stepI, stepG, stepK *)
Definition step M N := stepI M N \/ stepG M N \/ stepK M N.

(* reflexive, transitive closure of step *)
Notation steps := (clos_refl_trans tm step).
```

Original author formal source: stlc_facts.v

```
(*
  facts about the simplty typed lambda-calculus
*)

Require Import Lia List Relations PeanoNat.
Import ListNotations.

From NonIdempotent Require Import facts stlc.
Require Import ssreflect ssrbool.

Arguments nth_error_split {A l n a}.
Arguments in_split {A x}.

Scheme nf_ind' := Induction for nf Sort Prop
  with neutral_ind' := Induction for neutral Sort Prop.

Inductive stlc_app_spec (Gamma : list sty) (M N : tm) (t : sty) : Prop :=
  | stlc_app_spec_intro s : stlc Gamma M (sarr s t) -> stlc Gamma N s -> stlc_app_spec Gamma M N t.

Inductive stlc_lam_spec (Gamma : list sty) (t' : sty) (M : tm) (t : sty) : Prop :=
  | stlc_lam_spec_intro s u : stlc (s :: Gamma) M u -> t = sarr s u -> t = t' -> stlc_lam_spec Gamma t' M t.

Lemma stlcE Gamma M t : stlc Gamma M t ->
  match M with
  | var x => nth_error Gamma x = Some t
  | app M N => stlc_app_spec Gamma M N t
  | lam t' M => stlc_lam_spec Gamma t' M t
  end.
Proof.
  intros [].
  - done.
  - by econstructor; first by eassumption.
  - by econstructor; first by eassumption.
Qed.

Lemma ren_ext M xi1 xi2 :
  (forall x, xi1 x = xi2 x) ->
  ren xi1 M = ren xi2 M.
Proof.
  elim: M xi1 xi2=> /=.
  - move=> *. by congr var.
  - move=> ? IH1 ? IH2 > ?. congr app.
    + by apply: IH1.
    + by apply: IH2.
  - move=> ?? IH > ?. congr lam.
    apply: IH. move=> [|x] /=; by [|congr S].
Qed.

Lemma subst_ext M sigma1 sigma2 :
  (forall x, sigma1 x = sigma2 x) ->
  subst sigma1 M = subst sigma2 M.
Proof.
  elim: M sigma1 sigma2=> /=.
  - move=> >. apply.
  - move=> ? IH1 ? IH2 > ?. congr app.
    + by apply: IH1.
    + by apply: IH2.
  - move=> ?? IH > Hsigma. congr lam.
    apply: IH. move=> [|x] /=; by [|congr ren].
Qed.

Lemma ren_id M : ren id M = M.
Proof.
  elim: M=> /=.
  - done.
  - by move=> ? -> ? ->.
  - move=> ?? IH. congr lam. rewrite -[RHS]IH.
    apply: ren_ext. by case.
Qed.

Lemma subst_var M : subst var M = M.
Proof.
  elim: M=> /=.
  - done.
  - by move=> ? -> ? ->.
  - move=> ?? IH. congr lam. rewrite -[RHS]IH.
    apply: subst_ext. by case.
Qed.

Lemma ren_as_subst xi M : ren xi M = subst (funcomp var xi) M.
Proof.
  elim: M xi=> /=.
  - done.
  - move=> ? IH1 ? IH2 ?. by rewrite IH1 IH2.
```

```
  - move=> ?? IH ?. congr lam. rewrite IH.
    apply: subst_ext. by case.
Qed.

Lemma ren_ren xi1 xi2 M :
  ren xi2 (ren xi1 M) = ren (funcomp xi2 xi1) M.
Proof.
  elim: M xi1 xi2=> /=.
  - done.
  - move=> ? IH1 ? IH2 ??. by rewrite IH1 IH2.
  - move=> ?? IH ??. congr lam. rewrite IH.
    rewrite !ren_as_subst. apply: subst_ext. by case.
Qed.

Lemma ren_subst xi sigma M :
  ren xi (subst sigma M) = subst (funcomp (ren xi) sigma) M.
Proof.
  elim: M xi sigma=> /=.
  - done.
  - move=> ? IH1 ? IH2 ??. by rewrite IH1 IH2.
  - move=> ?? IH ??. congr lam. rewrite IH.
    apply: subst_ext.
    move=> [|x] /=; [done|].
    rewrite !ren_ren.
    rewrite !ren_as_subst. by apply: subst_ext.
Qed.

Lemma subst_ren xi sigma M :
  subst sigma (ren xi M) = subst (funcomp sigma xi) M.
Proof.
  elim: M xi sigma=> /=.
  - done.
  - move=> ? IH1 ? IH2 ??. by rewrite IH1 IH2.
  - move=> ?? IH ??. congr lam. rewrite IH.
    apply: subst_ext. by case.
Qed.

Lemma subst_subst sigma1 sigma2 M :
  subst sigma2 (subst sigma1 M) = subst (funcomp (subst sigma2) sigma1) M.
Proof.
  elim: M sigma1 sigma2=> /=.
  - done.
  - move=> ? IH1 ? IH2 ??. by rewrite IH1 IH2.
  - move=> ?? IH ??. congr lam. rewrite IH.
    apply: subst_ext=> - [|x] /=; [done|].
    rewrite ren_subst subst_ren.
    by apply: subst_ext.
Qed.

Lemma allfv_trivial (P : nat -> Prop) t : (forall x, P x) -> allfv P t.
Proof.
  elim: t P=> /=; [by auto..|].
  move=> ?? IH *. by apply: IH => - [|?].
Qed.

Lemma allfv_impl (P Q : nat -> Prop) t :
  (forall x, P x -> Q x) -> allfv P t -> allfv Q t.
Proof.
  elim: t P Q => /=.
  - move=> >. by apply.
  - move=> ? IH1 ? IH2 > /[dup] /IH1 {}IH1 /IH2 {}IH2. tauto.
  - move=> ? > IH > H /=. apply: IH.
    by case.
Qed.

Lemma allfv_allfv_impl (P Q : nat -> Prop) t :
  allfv (fun x => P x -> Q x) t -> allfv P t -> allfv Q t.
Proof.
  elim: t P Q => /=.
  - move=> >. by apply.
  - move=> ? IH1 ? IH2 > [/IH1 {}IH1] /IH2 {}IH2.
    by move=> [/IH1] {}IH1 /IH2 {}IH2.
  - move=> ? > IH > H /=. apply: IH.
    apply: allfv_impl H. by case.
Qed.

Lemma allfv_dec P M : (forall x, {P x} + {not (P x)}) -> {allfv P M} + {not (allfv P M)}.
Proof.
  elim: M P=> /=.
  - done.
  - move=> s IH1 t IH2 P /[dup] /(IH1 P) {}IH1 /(IH2 P) {}IH2. tauto.
  - move=> ?? IH P H.
    case: (IH (scons True P)); [|by auto..].
    move=> [|?] /=; by auto.
Qed.

Lemma allfv_ren (P : nat -> Prop) xi t :
```

```
  allfv (funcomp P xi) t -> allfv P (ren xi t).
Proof.
  elim: t P xi.
  - done.
  - move=> ? IH1 ? IH2 > /= [??]. by auto.
  - move=> ?? IH > /= H. apply: IH.
    by apply: allfv_impl H => - [|?].
Qed.

Lemma allfv_ren' (P : nat -> Prop) xi t :
  allfv P (ren xi t) -> allfv (funcomp P xi) t.
Proof.
  elim: t P xi.
  - done.
  - move=> ? IH1 ? IH2 > /= [??]. split.
    + by apply: IH1.
    + by apply: IH2.
  - move=> ?? IH > /= /IH.
    by apply: allfv_impl => - [|?].
Qed.

Lemma allfv_subst (P : nat -> Prop) sigma t :
  allfv (fun x => allfv P (sigma x)) t -> allfv P (subst sigma t).
Proof.
  elim: t P sigma.
  - done.
  - move=> ? IH1 ? IH2 > /= [??]. by auto.
  - move=> ?? IH > /= H. apply: IH.
    apply: allfv_impl H => - [|?] /= H; [done|].
    apply /allfv_ren. by apply: allfv_impl H.
Qed.

Lemma allfv_subst' (P : nat -> Prop) sigma t :
  allfv P (subst sigma t) -> allfv (fun x => allfv P (sigma x)) t.
Proof.
  elim: t P sigma.
  - done.
  - move=> ? IH1 ? IH2 > /= [??]. by auto.
  - move=> ?? IH > /= /IH.
    apply: allfv_impl => - [|?] /=; [done|].
    move=> /allfv_ren'. by apply: allfv_impl.
Qed.

Lemma ext_allfv_subst sigma1 sigma2 t : allfv (fun x=> sigma1 x = sigma2 x) t ->
  subst sigma1 t = subst sigma2 t.
Proof.
  elim: t sigma1 sigma2.
  - by move=> > /= ?.
  - by move=> ? IH1 ? IH2 ?? /= [/IH1 -> /IH2 ->].
  - move=> ?? IH > /= Hsigma. congr lam. apply: IH.
    apply: allfv_impl Hsigma.
    by move=> [|x] /= => [|->].
Qed.

Lemma ext_allfv_ren xi1 xi2 t : allfv (fun x=> xi1 x = xi2 x) t -> ren xi1 t = ren xi2 t.
Proof.
  move=> H. rewrite !ren_as_subst. apply: ext_allfv_subst.
  by apply: allfv_impl H => ? /= ->.
Qed.

Lemma allfv_list M : exists l, forall P, allfv P M <-> Forall P l.
Proof.
  elim: M.
  - move=> x. exists [x] => P /=. split.
    + move=> ?. by constructor.
    + by move=> /Forall_cons_iff [].
  - move=> > [l1 IH1] > [l2 IH2]. exists (l1 ++ l2) => P /=. split.
    + move=> [??]. apply /Forall_app. split.
      * by apply /IH1.
      * by apply /IH2.
    + by move=> /Forall_app [/IH1 ?] /IH2.
  - move=> > [l IH]. exists (concat (map (fun x => match x with 0 => [] | S x => [x] end) l)).
    move=> P /=. split.
    + move=> /IH. elim; first done.
      move=> [|x] ? /=; first done.
      by constructor.
    + move=> H. apply /IH. elim: l {IH} H; first done.
      move=> [|x] ? IH /=.
      * move=> ?. constructor; first done.
        by apply: IH.
      * move=> /Forall_cons_iff [??].
        constructor; first done.
        by apply: IH.
Qed.

Lemma stlc_allfv_ren Gamma Delta xi M t : allfv (fun x => forall s, nth_error Gamma x = Some s -> nth_error
     Delta (xi x) = Some s) M ->
```

```
  stlc Gamma M t -> stlc Delta (ren xi M) t.
Proof.
  move=> + H. elim: H xi Delta.
  - move=> > ? > /= H. constructor. by apply: H.
  - move=> > ? IH1 ? IH2 > /= [] /IH1 {}IH1 /IH2 {}IH2.
    econstructor; by eassumption.
  - move=> > ? IH > /= H. constructor.
    apply: IH. by apply: allfv_impl H => - [|?] /=.
Qed.

Lemma stlc_ren Gamma Delta xi M t : (forall x s, nth_error Gamma x = Some s -> nth_error Delta (xi x) = Some
      s) ->
  stlc Gamma M t -> stlc Delta (ren xi M) t.
Proof.
  move=> ?. apply: stlc_allfv_ren. by apply: allfv_trivial.
Qed.

Lemma stlc_inj Gamma M t1 t2 :
  stlc Gamma M t1 ->
  stlc Gamma M t2 ->
  t1 = t2.
Proof.
  move=> H. elim: H t2.
  - move=> > ?? /stlcE. congruence.
  - by move=> > ? IH ? _ ? /stlcE [] > /IH [].
  - move=> > ? IH ? /stlcE [] > + -> [??]. by subst.
Qed.

Lemma stlc_allfv_not_None Gamma M t :
  stlc Gamma M t ->
  allfv (fun x => nth_error Gamma x <> None) M.
Proof.
  elim.
  - by move=> > /= ->.
  - by move=> > /=.
  - move=> > /= ?. by apply: allfv_impl => -[|?].
Qed.

(* typing is preserved under substitutions *)
Theorem stlc_subst {Gamma Delta M t} sigma :
  stlc Gamma M t ->
  (forall n s, nth_error Gamma n = Some s -> stlc Delta (sigma n) s) ->
  stlc Delta (subst sigma M) t.
Proof.
  move=> H. elim: H Delta sigma.
  - move=> > ??? H /=. by apply: H.
  - move=> > ? IH1 ? IH2 > H /=. econstructor.
    + by apply: IH1.
    + by apply: IH2.
  - move=> > ? IH > H /=. constructor.
    apply: IH=> - [|n] /=.
    + move=> ? [<-]. by constructor.
    + move=> ? /H ?. by apply: stlc_ren; last by eassumption.
Qed.

Theorem stepI_reduction M N Gamma t : stepI M N -> stlc Gamma M t -> stlc Gamma N t.
Proof.
  move=> H. elim: H Gamma t.
  - move=> > ? >.
    move=> /stlcE [] ? + HN.
    move=> /stlcE [] ?? HM [??] ?. subst.
    apply: stlc_subst; first by eassumption.
    move=> [|n] ? /=; first by congruence.
    move=> ?. by constructor.
  - move=> > ? IH >.
    move=> /stlcE [] *. subst.
    constructor. by apply: IH.
  - move=> > ? IH >.
    move=> /stlcE [] *. subst.
    econstructor; [|by eassumption].
    by apply: IH.
  - move=> > ? IH >.
    move=> /stlcE [] *. subst.
    econstructor; [by eassumption|].
    by apply: IH.
Qed.

Theorem stepK_reduction M N Gamma t : stepK M N -> stlc Gamma M t -> stlc Gamma N t.
Proof.
  move=> H. elim: H Gamma t.
  - move=> > _ > /stlcE [] ?.
    move=> /stlcE [] ?? + [??] ??. subst.
    move=> /(stlc_allfv_ren _ _ (Nat.pred)).
    rewrite ren_ren ren_id. apply.
    apply: allfv_ren. by apply: allfv_trivial => ? /=.
  - move=> > ? IH >.
    move=> /stlcE [] *. subst.
```

```
    constructor. by apply: IH.
  - move=> > ?? IH >.
    move=> /stlcE [] *. subst.
    by econstructor; [eassumption|apply: IH].
  - move=> > ? IH >.
    move=> /stlcE [] *. subst.
    by econstructor; [eassumption|apply: IH].
  - move=> > ? IH >.
    move=> /stlcE [] *. subst.
    by econstructor; [apply: IH|eassumption].
Qed.

Theorem stepG_reduction M N Gamma t : stepG M N -> stlc Gamma M t -> stlc Gamma N t.
Proof.
  move=> H. elim: H Gamma t.
  - move=> > /stlcE [] ?.
    move=> /stlcE [] ??.
    move=> /stlcE [] ??? -> [??] [??] [??] ?. subst.
    econstructor. econstructor.
    + econstructor. apply: stlc_ren; last by eassumption.
      move=> [|[|?] ?] /=; congruence.
    + by apply: stlc_ren; last by eassumption.
  - move=> > ? IH >.
    move=> /stlcE [] *. subst.
    constructor. by apply: IH.
  - move=> > ? IH >.
    move=> /stlcE [] *. subst.
    by econstructor; [apply: IH|eassumption].
  - move=> > ? IH >.
    move=> /stlcE [] *. subst.
    by econstructor; [eassumption|apply: IH].
Qed.

(* subject reduction wrt. stepI, stepG, stepK *)
Theorem stlc_reduction M N Gamma t : step M N -> stlc Gamma M t -> stlc Gamma N t.
Proof.
  by move=> [/stepI_reduction|[/stepG_reduction|/stepK_reduction]]; apply.
Qed.

Fixpoint size_sty (t : sty) : nat :=
  match t with
  | satom _ => 1
  | sarr s t => 1 + size_sty s + size_sty t
  end.

Fixpoint size_tm (M : tm) :=
  match M with
  | var _ => 1
  | app M N => S (size_tm M) + (size_tm N)
  | lam _ M => S (size_tm M)
  end.

Lemma size_tm_ren xi M : size_tm (ren xi M) = size_tm M.
Proof.
  elim: M xi.
  - done.
  - move=> > IH1 ? IH2 ? /=. by rewrite IH1 IH2.
  - move=> > IH ? /=. by rewrite IH.
Qed.

Lemma allfv_stepI P M N : stepI M N -> allfv P M -> allfv P N.
Proof.
  move=> H. elim: H P.
  - move=> > ?? /= [HM ?]. apply: allfv_subst.
    by apply: allfv_impl HM=> - [|x].
  - move=> > ? IH > /=. by apply: IH.
  - move=> > ? IH > /= []. by auto.
  - move=> > ? IH > /= []. by auto.
Qed.

Lemma allfv_stepI' P M N : stepI M N -> allfv P N -> allfv P M.
Proof.
  move=> H. elim: H P.
  - move=> ? {}M {}N H1M P /= /allfv_subst' /= H2M. split.
    + by apply: allfv_impl H2M=> - [|x] /=.
    + have [l Hl] := allfv_list M.
      have : In 0 l.
      { have [|H] := In_dec Nat.eq_dec 0 l; first done.
        exfalso. apply: H1M. apply /Hl.
        elim: (l) H; first done.
        move=> [|?] > IH /= /Decidable.not_or [?] /IH ?; first done.
        by constructor. }
      move=> /in_split [?] [?] ?. subst l.
      by move: H2M => /Hl /Forall_app [?] /Forall_cons_iff [] /=.
  - move=> > ? IH > /=. by apply: IH.
  - move=> > ? IH > /= []. by auto.
  - move=> > ? IH > /= []. by auto.
```

```
Qed.

Lemma allfv_stepG P M N : stepG M N -> allfv P M -> allfv P N.
Proof.
  move=> H. elim: H P.
  - move=> > /= [HM HN]. split.
    + apply: allfv_ren.
      by apply: allfv_impl HM=> - [|[|x]] /=.
    + by apply: allfv_ren.
  - move=> > ? IH > /=. by apply: IH.
  - move=> > ? IH > /= []. by auto.
  - move=> > ? IH > /= []. by auto.
Qed.

Lemma allfv_stepG' P M N : stepG M N -> allfv P N -> allfv P M.
Proof.
  move=> H. elim: H P.
  - move=> > /= [/allfv_ren' H2M1] /allfv_ren' H2M2. split.
    + by apply: allfv_impl H2M1=> - [|[|?]] /=.
    + by apply: allfv_impl H2M2=> - [|?] /=.
  - move=> > ? IH > /=. by apply: IH.
  - move=> > ? IH > /= []. by auto.
  - move=> > ? IH > /= []. by auto.
Qed.
```

Original author formal source: stlc_nf.v

```
(*
  normalization property of simply typed terms:
  if M is simply typed, then M (I, gamma)-reduces to some term, which K-reduces to a normal form
*)

Require Import Lia List Relations PeanoNat Permutation Sorting.Heap Wellfounded.
Import ListNotations.

From NonIdempotent Require Import facts stlc stlc_facts.
Require Import ssreflect ssrbool.

#[local] Arguments Permutation_in {A l l' x}.

Lemma has_var_zero_dec M : {has_var_zero M} + {not (has_var_zero M)}.
Proof.
  have /(allfv_dec _ M) [?|?] : forall x, {is_nonzero x} + {not (is_nonzero x)}.
  { case; tauto. }
  - right=> H. by apply: H.
  - by left.
Qed.

Definition inc_depth : nat * nat -> nat * nat :=
  fun '(n, m) => (n, S m).

(* list of sizes of I, G redexes together with depth wrt. abstraction *)
Fixpoint measure (M : tm) : list (nat * nat) :=
  match M with
  | var _ => []
  | lam _ M => map inc_depth (measure M)
  | app M1 M2 =>
    match M1 with
    | var _ => []
    | app _ _ => []
    | lam t M1' =>
      if has_var_zero_dec M1' then [(size_sty t, 0)] (* top-level I-redex *)
      else
        match M1' with
        | var _ => []
        | app _ _ => []
        | lam t' _ => [(1 + size_sty t', 0)] (* top-level G-redex *)
        end
    end ++ measure M1 ++ measure M2
  end.

Arguments measure : simpl never.

Lemma measure_app_lam M N t : has_var_zero M -> measure (app (lam t M) N) = [(size_sty t, 0)] ++ map
     inc_depth (measure M) ++ measure N.
Proof.
  rewrite /measure -/measure /=.
  by case: (has_var_zero_dec M)=> /=.
Qed.

Lemma measure_app_lam' M N t : not (has_var_zero M) -> measure (app (lam t M) N) =
  match M with
  | var _ => []
  | app _ _ => []
  | lam t3 _ => [(1 + size_sty t3, 0)] (* G-redex *)
  end ++ map inc_depth (measure M) ++ measure N.
Proof.
  rewrite /measure -/measure /=.
  by case: (has_var_zero_dec M).
Qed.

Lemma measure_var x : measure (var x) = [].
Proof. done. Qed.

Lemma measure_lam t M : measure (lam t M) = map inc_depth (measure M).
Proof. done. Qed.

Lemma measure_app_var x M : measure (app (var x) M) = measure M.
Proof. done. Qed.

Lemma measure_app_app M1 M2 M3 : measure (app (app M1 M2) M3) = measure (app M1 M2) ++ measure M3.
Proof. done. Qed.

Lemma measure_ren xi M : measure (ren xi M) = measure M.
Proof.
  elim: M xi.
  - move=> x xi /=. by rewrite !measure_var.
  - move=> [x|M1 M2|t' M] IH1 N IH2 xi /=.
    + rewrite !measure_app_var. by apply: IH2.
    + rewrite !measure_app_app IH2.
      congr List.app. by apply: IH1.
```

```
    + case: (has_var_zero_dec M).
      * move=> /[dup] H1M /measure_app_lam ->.
        have /measure_app_lam -> : has_var_zero (ren (scons 0 (fun x : nat => S (xi x))) M).
        { move=> /allfv_ren' H. apply: H1M.
          by apply: allfv_impl H=> - [|x]. }
        congr cons. rewrite IH2.
        congr List.app. by apply: IH1.
      * move=> /[dup] H1M /measure_app_lam' ->.
        have /measure_app_lam' -> : not (has_var_zero (ren (scons 0 (fun x : nat => S (xi x))) M)).
        { move=> H. apply: H1M=> H'. apply: H. apply: allfv_ren.
          by apply: allfv_impl H'=> - [|x]. }
        rewrite IH2.
        have /= := IH1 xi. rewrite !measure_lam=> ->.
        by move: (M) => [[|?]|??|??] /=.
  - move=> > IH xi /=. rewrite !measure_lam.
    congr map. by apply: IH.
Qed.

Lemma measure_app_stlc Gamma M N t :
  stlc Gamma M t ->
  exists ns,
    Permutation (measure (app M N)) (ns ++ measure M ++ measure N) /\
    Forall (fun '(n, m) => n <= size_sty t) ns.
Proof.
  move=> H. rewrite /measure -/measure. eexists. split.
  { rewrite !app_assoc. by do 2 apply: Permutation_app_tail. }
  case: H; [done..|].
  move=> {}M s'' t''. case: (has_var_zero_dec M); first by constructor.
  case: M=> /=; [done..|].
  move=> > ? /stlcE [>] *. subst.
  constructor; last done.
  case: (s'')=> /=; lia.
Qed.

Lemma measure_app_ren xi M1 M2 : measure (app (ren xi M1) M2) = measure (app M1 M2).
Proof.
  rewrite /measure -/measure. rewrite !measure_ren. congr List.app.
  case: M1=> /=; [done..|].
  move=> ? M. case: (has_var_zero_dec M).
  + move=> H.
    have : has_var_zero (ren (scons 0 (fun x : nat => S (xi x))) M).
    { move=> /allfv_ren' H'. apply: H.
      apply: allfv_impl H'. by case. }
    by case: (has_var_zero_dec _).
  + move=> H.
    have : not (has_var_zero (ren (scons 0 (fun x : nat => S (xi x))) M)).
    { move=> H'. apply: H=> H.
      apply: H'. apply: allfv_ren.
      apply: allfv_impl H. by case. }
    case: (has_var_zero_dec _); first done.
    move=> /=. by case: (M).
Qed.

(* measure decreases on innermost I-redex contraction *)
Lemma measure_decrease_stepIRed Gamma M N s t :
  stlc (s :: Gamma) M t ->
  stlc Gamma N s ->
  has_var_zero M -> measure N = [] ->
  exists ns,
    Permutation (measure (subst (scons N var) M))
      (ns ++ measure M ++ measure N) /\
    Forall (fun '(n, m) => n < size_sty (sarr s t)) ns.
Proof.
  move=> H2M H2N H1M H1N. rewrite H1N app_nil_r.
  have HNM : stlc Gamma (subst (scons N var) M) t.
  { apply: stlc_subst; first by eassumption.
    by move=> [|x] ? /=; [congruence|apply: stlc_var]. }
  have :
    match N with
    | var _ => True
    | app _ _ => True
    | lam t' _ => t' = s
    end.
  { case: (N) H2N; [done..|].
    by move=> > /stlcE []. }
  move: H1M H1N H2N. clear.
  have Hst : S (size_sty s) < size_sty s + S (size_sty t) by case: (t); cbn; lia.
  move=> _ H2N H1N H3N.
  rewrite -(ren_id N). move: (id).
  elim /(Nat.measure_induction _ size_tm): M. case.
  - move=> [|x] _ /= xi; rewrite !measure_var.
    + rewrite measure_ren H2N. exists nil. by constructor.
    + exists nil. by constructor.
  - move=> M M3 IH.
    have /IH {}IH1 : size_tm M < size_tm (app M M3) by cbn; lia.
    have /IH {}IH2 : size_tm M3 < size_tm (app M M3) by cbn; lia.
    move: M {IH} IH1=> [[|x]|M1 M2|t'' M] IH1 xi /=.
```

```
    + rewrite measure_app_var measure_app_ren.
      move: H1N=> /measure_app_stlc=> /(_ (subst (scons (ren xi N) var) M3)).
      move=> [n's] [+ H2n's].
      move: (IH2 xi)=> /= [ns] [].
      move: (subst _ _)=> M'3. move: (app N M'3)=> N'. rewrite H2N /=.
      move=> *. exists (n's ++ ns). split.
      * apply: Permutation_trans; first by eassumption.
        rewrite -app_assoc. by apply: Permutation_app_head.
      * apply /Forall_app. split; last done.
        apply: Forall_impl H2n's=> - [??]. lia.
    + rewrite !measure_app_var. apply: IH2.
    + rewrite !measure_app_app.
      move: (IH1 xi) (IH2 xi)=> /= [n1s] [H1n1s H2n1s] [n2s] [H1n2s H2n2s].
      exists (n1s ++ n2s). split.
      * apply: Permutation_trans. { apply: Permutation_app; by eassumption. }
        move: (measure (app _ _))=> ?.
        rewrite !app_assoc. apply Permutation_app_tail.
        rewrite -!app_assoc. apply Permutation_app_head.
        by apply: Permutation_app_comm.
      * apply /Forall_app. by split.
    + case: (has_var_zero_dec M).
      { move=> H.
        have : has_var_zero (subst (scons (var 0) (fun x => ren S (scons (ren xi N) var x))) M).
        { move=> /allfv_subst' H'. apply: H.
          apply: allfv_impl H'. by case. }
        move: H => /measure_app_lam -> /measure_app_lam ->.
        move: (IH2 xi) (IH1 xi)=> /=. rewrite !measure_lam.
        move=> [n1s] [+ H2n1s] [n2s] [+ H2n2s].
        move: (measure (subst _ _)) (measure (subst _ _))=> ????.
        exists (n2s ++ n1s). split.
        - apply: (Permutation_app_middle [(size_sty t'', 0)] nil _ (n2s ++ n1s) _).
          move=> /=. apply: Permutation_trans. { apply: Permutation_app; by eassumption. }
          rewrite -!app_assoc. apply: Permutation_app_head.
          rewrite !app_assoc. apply: Permutation_app_tail. by apply: Permutation_app_comm.
        - apply /Forall_app. by split. }
      move=> H.
      have : not (has_var_zero (subst (scons (var 0) (fun x => ren S (scons (ren xi N) var x))) M)).
      { move=> H'. apply: H=> H. apply: H'. apply: allfv_subst.
        apply: allfv_impl H. case; first done.
        move=> ??. apply: allfv_ren. by apply: allfv_trivial. }
      move: H => /measure_app_lam' -> /measure_app_lam' ->.
      move: (IH2 xi) {IH1} (IH1 xi) => [n2s] [+ H2n2s] [n1s] [+ H2n1s].
      rewrite /= !measure_lam.
      { move: M => [[|[|x]]|??|??] /=.
        - move=> *. by exists n2s.
        - move=> *.
          exists (match ren S (ren xi N) with lam t3 _ => [(1 + size_sty t3, 0)] | _ => [] end ++ n1s ++ n2s
   ).
          split.
          + apply: Permutation_trans.
            { apply: Permutation_app; [by apply: Permutation_refl|].
              apply: Permutation_app; by eassumption. }
            rewrite !app_assoc app_nil_r. by apply: Permutation_app_tail.
          + apply /Forall_app. split; [|by apply /Forall_app].
            case: H1N; [done..|].
            move=> > /= ?. by constructor; [lia|].
        - move=> *. by exists n2s.
        - move: (measure (app _ _)) (measure (app _ _))=> *.
          exists (n1s ++ n2s). split.
          + apply: Permutation_trans. { apply: Permutation_app; by eassumption. }
            rewrite -!app_assoc. apply: Permutation_app_head.
            rewrite !app_assoc. apply: Permutation_app_tail. by apply: Permutation_app_comm.
          + apply /Forall_app. by split.
        - rewrite !measure_lam. move: (measure (subst _ _)) (measure (subst _ _))=> *.
          exists (n1s ++ n2s). split.
          + apply: (Permutation_app_middle [_] nil _ (n1s ++ n2s) _).
            move=> /=. apply: Permutation_trans. { apply: Permutation_app; by eassumption. }
            rewrite -!app_assoc. apply: Permutation_app_head.
            rewrite !app_assoc. apply: Permutation_app_tail. by apply: Permutation_app_comm.
          + apply /Forall_app. by split. }
  - move=> t' M IH xi /=.
    have -> : subst (scons (var 0) (fun x : nat => ren S (scons (ren xi N) var x))) M =
      subst (scons (ren S (ren xi N)) var) (ren swap M).
    { rewrite subst_ren. by apply: subst_ext=> - [|[|x]] /=. }
    have : size_tm (ren swap M) < size_tm (lam t M).
    { by rewrite size_tm_ren /=. }
    move=> /IH /(_ ((funcomp S xi))).
    rewrite !measure_lam measure_ren ren_ren.
    move=> [ns] [? Hns]. exists (map inc_depth ns). split.
    + rewrite -map_app. by apply: Permutation_map.
    + apply /Forall_map. apply: Forall_impl Hns. by case.
Qed.

(* rightmost lambda-I reduction *)
Inductive inner_stepI : (nat*nat) -> tm -> tm -> Prop :=
  | inner_stepIRed s t M N     : has_var_zero M -> measure N = [] ->
      inner_stepI (size_sty (sarr s t), 0) (app (lam (sarr s t) M) N) (subst (scons N var) M)
```

```
  | inner_stepILam t M M' n m    : inner_stepI (n, m) M M' -> inner_stepI (n, S m) (lam t M) (lam t M')
  | inner_stepIAppL M M' N nm : inner_stepI nm M M' -> inner_stepI nm (app M N) (app M' N)
  | inner_stepIAppR M N N' nm : inner_stepI nm N N' -> inner_stepI nm (app M N) (app M N').

Lemma inner_stepI_stepI n M N : inner_stepI n M N -> stepI M N.
Proof.
  by elim; auto using stepI.
Qed.

Lemma Forall2_le_refl (l : list (nat*nat)) : Forall2 (fun '(n1, m1) '(n2, m2) => n1 = n2 /\ m2 <= m1) l l.
Proof.
  elim: l; first done.
  move=> [??] ??. by constructor; [lia|].
Qed.

#[local] Hint Resolve Forall2_le_refl : core.

(* measure decreases on rightmost I-redex contraction *)
Lemma measure_decrease_inner_stepI Gamma n M1 M2 t :
  inner_stepI n M1 M2 ->
  stlc Gamma M1 t ->
  exists n1s, Permutation ([n] ++ n1s) (measure M1) /\
    exists n2s, Forall2 (fun '(n1, m1) '(n2, m2) => n1 = n2 /\ m2 <= m1) n1s n2s /\
      exists ns, Permutation (ns ++ n2s) (measure M2) /\ Forall (fun '(n', m') => n' < fst n \/ n' = fst n
    /\ m' < snd n) ns.
Proof.
  move=> H. elim: H Gamma t.
  - move=> s t {}M1 {}M2 H1M1 H2M1 Gamma t'.
    move=> /stlcE [] s' /stlcE [] s'' t'' H3M1 [??] [??] H3M2. subst.
    move: (H1M1)=> /measure_app_lam ->.
    move: H1M1 H2M1 H3M1 H3M2=> /measure_decrease_stepIRed /[apply] /[apply] /[apply].
    move=> [ns] [? Hns]. exists ((map inc_depth (measure M1)) ++ (measure M2)). split; first done.
    exists (measure M1 ++ measure M2). split.
    { apply: Forall2_app; last done.
      apply: Forall2_map_l. apply: Forall2_eq.
      apply /Forall_forall=> - [??] /=. lia. }
    exists ns. split; first by apply: Permutation_sym.
    apply: Forall_impl Hns => - [??] /=. lia.
  - move=> > ? IH > /stlcE [] ??.
    move=> /IH [n1s] [Hn1s] [n2s] [Hn1sn2s] [ns] [Hn2s Hns] ??. subst.
    rewrite !measure_lam.
    exists (map inc_depth n1s). split.
    { rewrite -/(map inc_depth [(_, _)]) -!map_app. by apply: Permutation_map. }
    exists (map inc_depth n2s). split.
    { apply: Forall2_map_l. apply: Forall2_map_r.
      apply: Forall2_impl Hn1sn2s=> - [??] [??] /=. lia. }
    exists (map inc_depth ns). split.
    { rewrite -!map_app. by apply: Permutation_map. }
    apply /Forall_map. apply: Forall_impl Hns=> - [??] /=. lia.
  - move=> {}M1 {}M2 M3 {}n Hn IH Gamma t.
    move=> /stlcE [] s' /[dup] H1M1.
    move=> /IH [n1s] [Hn1s] [n2s] [Hn1sn2s] [ns] [Hn2s Hns] ?.
    move: Hn Hn1s Hn2s H1M1 Hns Hn1sn2s. case; clear.
    + move=> s'' t'' M1 M2 H1M1 H1M2.
      rewrite !measure_app_app.
      move: (H1M1)=> /measure_app_lam ->.
      rewrite H1M2 !app_nil_r.
      move=> Hn1s Hn2s /stlcE [] s''' /stlcE [] s'''' t'''' H3M1 [??] [??] H3M2. subst.
      move=> ??. exists (n1s ++ measure M3). split.
      { rewrite !app_assoc. by apply: Permutation_app_tail. }
      exists (n2s ++ measure M3). split; first by apply: Forall2_app.
      move: M1 H1M1 H3M1 Hn1s Hn2s.
      { move=> [[|x]|t'' M'1 M'2|t'' M'1] /=.
        - move=> _ /stlcE [] /= ? Hn1s Hn2s. subst.
          move: H3M2=> /measure_app_stlc => /(_ M3) [n's].
          move: (measure (app _ _))=> ? [H1n's H2n's].
          exists (n's ++ ns). split.
          { apply: Permutation_trans; last by (apply: Permutation_sym; eassumption).
            rewrite !app_assoc. apply: Permutation_app_tail.
            rewrite -!app_assoc. by apply: Permutation_app. }
          apply /Forall_app. split; last done.
          apply: Forall_impl H2n's=> - [??] /=. lia.
        - done.
        - move=> ?. rewrite !measure_app_app.
          move: (measure (app _ _)) (measure (app _ _))=> *.
          exists ns. split; last done.
          rewrite !app_assoc. by apply: Permutation_app_tail.
        - move=> H1M'1 H2M'1. rewrite !measure_lam.
          have ? : t'' = sarr s' t.
          { by move: H2M'1 => /stlcE []. }
          subst t''.
          have : stlc Gamma (lam (sarr s' t) (subst (scons (var 0) (fun x : nat => ren S (scons M2 var x)))
     M'1)) (sarr s' t).
          { rewrite -/(subst _ (lam _ _)).
            apply: stlc_subst; first by eassumption.
            move=> [|?] ? /=; first by congruence.
            by move=> /stlc_var. }
```

```
      move=> /measure_app_stlc=> /(_ M3) [n's] [+ H2n's].
      rewrite !measure_lam. move: (measure (app _ _)) (measure (subst _ _))=> *.
      exists (n's ++ ns). split.
      { apply: Permutation_trans; last by (apply: Permutation_sym; eassumption).
        rewrite !app_assoc. apply: Permutation_app_tail.
        rewrite -!app_assoc. by apply: Permutation_app. }
      apply /Forall_app. split; last done.
      apply: Forall_impl H2n's=> - [??] /=. lia. }
+ move=> t' M1 M2 n m HM1M2. rewrite !measure_lam.
  case: (has_var_zero_dec M1).
  * move=> H3M1. have : has_var_zero M2.
    { move=> H3M2. apply: H3M1.
      apply: allfv_stepI'; [apply: inner_stepI_stepI|]; by eassumption. }
    move=> /measure_app_lam ->. move: H3M1=> /measure_app_lam ->.
    move=> Hn1s Hn2s /stlcE [>] ? [??] ? Hns Hn1sn2s. subst.
    eexists (n1s ++ _ ++ measure M3). split.
    { rewrite !app_assoc. apply: Permutation_app_tail.
      apply: Permutation_trans; last by apply: Permutation_app_comm.
      by apply: Permutation_app. }
    eexists (n2s ++ _ ++ measure M3). split; first by apply: Forall2_app.
    exists ns. split; last done.
    { rewrite !app_assoc. apply: Permutation_app_tail.
      apply: Permutation_trans; last by apply: Permutation_app_comm.
      by apply: Permutation_app. }
  * move=> H3M1. have : not (has_var_zero M2).
    { move=> H3M2. apply: H3M1=> H3M1. apply: H3M2.
      apply: allfv_stepI; [apply: inner_stepI_stepI|]; by eassumption. }
    move=> /measure_app_lam' ->. move: H3M1=> /measure_app_lam' ->.
    { move E: (n, m) HM1M2=> nm H. case: H E.
      - move=> s'' t'' M'1 N /[dup] /measure_app_lam -> ++ [??]. subst.
        move: M'1=> [[|x]|M'1 M'2|? M'1] /=.
        + move=> _ HN Hn1s Hn2s.
          move=> /stlcE [] ?? + [??] ?. subst.
          move=> /stlcE [] ? ++.
          move=> /stlcE [] ?? + [??] [??]. subst.
          move=> /stlcE [?]. subst.
          move: Hn1s Hn2s. rewrite HN=> H. clear HN.
          move=> ? HN *.
          exists (measure M3). split; first done.
          exists (measure M3). split; first done.
          move: N HN => [?|??|? N]; [by exists nil..|].
          move=> /stlcE [>] *. subst.
          eexists [_]. split; first done.
          constructor; last done.
          move=> /=. lia.
        + done.
        + move=> _ ?? H ???.
          exists (n1s ++ measure M3). split.
          { rewrite !cons_app !app_assoc. by apply: Permutation_app. }
          exists (n2s ++ measure M3). split; first by apply: Forall2_app.
          exists ns. split; last done.
          rewrite !app_assoc. by apply: Permutation_app.
        + rewrite !measure_lam. move: (measure (subst _ _))=> ???? H.
          move=> /stlcE [] ?? + [??] ?. subst.
          move=> /stlcE [] ? ++.
          move=> /stlcE [] ?? + [??] [??]. subst.
          move=> /stlcE [] ?? + ??. subst=> ????.
          exists (n1s ++ measure M3). split.
          { rewrite !cons_app !app_assoc. by apply: Permutation_app_tail. }
          exists (n2s ++ measure M3). split; first by apply: Forall2_app.
          eexists ([_] ++ ns). split.
          { rewrite !cons_app !app_assoc. apply: Permutation_app_tail.
            rewrite -!app_assoc. by apply: Permutation_app. }
          apply /Forall_app. split; last done.
          constructor; last done.
          move: (s'')=> [?|??] /=; lia.
      - move=> >. rewrite !measure_lam=> ? [??]. subst.
        move=> Hn1s Hn2s ? Hns ?.
        eexists (n1s ++ [_] ++ measure M3). split.
        { rewrite !app_assoc. apply: Permutation_app_tail.
          apply: Permutation_trans; last by apply: Permutation_app_comm.
          by apply: Permutation_app. }
        eexists (n2s ++ [_] ++ measure M3). split; first by apply: Forall2_app.
        exists ns. split; last done.
        rewrite !app_assoc. apply: Permutation_app_tail.
        apply: Permutation_trans; last by apply: Permutation_app_comm.
        by apply: Permutation_app.
      - move=> > ?? Hn1s Hn2s ? Hns ?. subst.
        exists (n1s ++ measure M3). split.
        { rewrite !app_assoc. by apply: Permutation_app. }
        exists (n2s ++ measure M3). split; first by apply: Forall2_app.
        exists ns. split; last done.
        rewrite !app_assoc. by apply: Permutation_app.
      - move=> > ?? Hn1s Hn2s ? Hns ?. subst.
        exists (n1s ++ measure M3). split.
        { rewrite !app_assoc. by apply: Permutation_app. }
        exists (n2s ++ measure M3). split; first by apply: Forall2_app.
```

```
            exists ns. split; last done.
            rewrite !app_assoc. by apply: Permutation_app. }
    + move=> M1 M2 N n ? Hn1s Hn2s ? Hns Hn1sn2s.
      rewrite !measure_app_app.
      exists (n1s ++ measure M3). split.
      { rewrite !app_assoc. by apply: Permutation_app. }
      exists (n2s ++ measure M3). split; first by apply: Forall2_app.
      exists ns. split; last done.
      rewrite !app_assoc. by apply: Permutation_app.
    + move=> > ? Hn1s Hn2s ? Hns Hn1sn2s. rewrite !measure_app_app.
      exists (n1s ++ measure M3). split.
      { rewrite !app_assoc. by apply: Permutation_app. }
      exists (n2s ++ measure M3). split; first by apply: Forall2_app.
      exists ns. split; last done.
      rewrite !app_assoc. by apply: Permutation_app.
  - move=> > ? IH > /stlcE [>] ?.
    move=> /IH [n1s] [Hn1s] [n2s] [Hn1sn2s] [ns] [Hn2s Hns].
    rewrite /measure -/measure. eexists (n1s ++ _). split.
    { rewrite !app_assoc. apply: Permutation_trans; first by apply: Permutation_app_comm.
      by apply: Permutation_app. }
    eexists (n2s ++ _ ++ measure _). split; first by apply: Forall2_app.
    eexists ns. split; last done.
    rewrite !app_assoc.
    apply: Permutation_trans; last by apply: Permutation_app_comm.
    rewrite !app_assoc. apply: Permutation_app; last done.
    by apply: Permutation_app.
Qed.

(* rightmost lambda-G reduction *)
Inductive inner_stepG : (nat*nat) -> tm -> tm -> Prop :=
  | inner_stepGRed s1 s2 t M N : not (has_var_zero (lam (sarr s2 t) M)) ->
    measure N = [] ->
    inner_stepG (1 + size_sty (sarr s2 t), 0)
      (app (lam (sarr s1 (sarr s2 t)) (lam (sarr s2 t) M)) N)
      (lam (sarr s2 t) (app (lam (sarr s1 t) (ren swap M)) (ren S N)))
  | inner_stepGLam t M M' n m     : inner_stepG (n, m) M M' -> inner_stepG (n, S m) (lam t M) (lam t M')
  | inner_stepGAppL M M' N nm  : inner_stepG nm M M' -> inner_stepG nm (app M N) (app M' N)
  | inner_stepGAppR M N N' nm  : inner_stepG nm N N' -> inner_stepG nm (app M N) (app M N').

Lemma inner_stepG_stepG n M N : inner_stepG n M N -> stepG M N.
Proof.
  by elim; auto using stepG.
Qed.

(* measure decreases on rightmost G-redex contraction *)
Lemma measure_decrease_inner_stepG Gamma n M1 M2 t :
  inner_stepG n M1 M2 ->
  stlc Gamma M1 t ->
  exists n1s, Permutation ([n] ++ n1s) (measure M1) /\
    exists n2s, Forall2 (fun '(n1, m1) '(n2, m2) => n1 = n2 /\ m2 <= m1) n1s n2s /\
      exists ns, Permutation (ns ++ n2s) (measure M2) /\ Forall (fun '(n', m') => n' < fst n \/ n' = fst n
     /\ m' < snd n) ns.
Proof.
  move=> H. elim: H Gamma t.
  - move=> s1 s2 t {}M1 {}M2 H1M1 H1M2 Gamma t'.
    move=> /stlcE [>] /stlcE [>] /stlcE [>] H2M1 ?. subst.
    move=> [??]. subst=> - [??]. subst=> - [?] H2M2. subst.
    rewrite measure_lam.
    have /measure_app_lam' -> : not (has_var_zero (ren swap M1)).
    { move=> H. apply: H1M1=> /= H1M1. apply: H.
      apply: allfv_ren. by apply: allfv_impl H1M1=> - [|[|?]] /=. }
    move: H1M1 => /measure_app_lam' ->. rewrite measure_lam !measure_ren H1M2 !app_nil_r.
    eexists. split; first done.
    rewrite map_app.
    eexists. split; first done.
    eexists. split; first done.
    apply /Forall_map.
    move: M1 H2M1=> [?|???|??] /=; [done..|].
    move=> /stlcE [>] ???. subst.
    constructor; last done.
    move=> /=. lia.
  - move=> > ? IH > /stlcE [] ?? + ?.
    move=> /IH [n1s] [Hn1s] [n2s] [Hn1sn2s] [ns] [Hn2s Hns].
    rewrite !measure_lam. exists (map inc_depth n1s). split.
    { rewrite -/(map inc_depth [(_, _)]) -map_app. by apply /Permutation_map. }
    exists (map inc_depth n2s). split.
    { apply: Forall2_map_l. apply: Forall2_map_r.
      apply: Forall2_impl Hn1sn2s=> - [??] [??] /=. lia. }
    exists (map inc_depth ns). split.
    { rewrite -map_app. by apply /Permutation_map. }
    apply /Forall_map. apply: Forall_impl Hns=> - [??] /=. lia.
  - move=> {}M1 {}M2 M3 {}n Hn IH Gamma t.
    move=> /stlcE [>] + ?.
    move=> /IH [n1s] [Hn1s] [n2s] [Hn1sn2s] [ns] [Hn2s Hns].
    case: Hn {IH} Hn1s Hn2s Hns.
    + move=> s1 s2 t' M'1 M'2 HM'1 HM'2.
      move: (HM'1) => /measure_app_lam' ->.
```

```
  rewrite !measure_app_app !measure_lam.
  have H'M'1 : ~ has_var_zero (ren swap M'1).
  { move=> H'M'1. apply: HM'1=> /= HM'1.
    apply: H'M'1. apply: allfv_ren.
    by apply: allfv_impl HM'1=> - [||[|?]]. }
  move: (H'M'1) => /measure_app_lam' ->.
  rewrite !measure_ren=> Hn1s H1ns H2ns.
  move: (HM'1) => /measure_app_lam' ->.
  rewrite !measure_lam. exists (n1s ++ measure M3). split.
  { rewrite !app_assoc. apply: Permutation_app_tail.
    by rewrite -!app_assoc. }
  exists (n2s ++ measure M3). split; first by apply: Forall2_app.
  have [H''M'1|H''M'1] := has_var_zero_dec M'1.
  { have /measure_app_lam -> : has_var_zero (app (lam (sarr s1 t') (ren swap M'1)) (ren S M'2)).
    { move=> /= [/allfv_ren' H _].
      apply: H''M'1. by apply: allfv_impl H=> - [||[|?]]. }
    move: H'M'1 => /measure_app_lam' ->.
    rewrite !measure_ren.
    exists ([(size_sty (sarr s2 t'), 0)] ++ ns). split.
    { rewrite !app_assoc. apply: Permutation_app_tail.
      rewrite -!app_assoc. by apply: Permutation_app. }
    apply /Forall_app. split; last done.
    constructor; last done.
    move=> /=. lia. }
  have /measure_app_lam' -> : not (has_var_zero (app (lam (sarr s1 t') (ren swap M'1)) (ren S M'2))).
  { move=> /= H. apply: H''M'1 => H''M'1.
    apply: H. split.
    - apply: allfv_ren. by apply: allfv_impl H''M'1=> - [|[|?]].
    - apply: allfv_ren. by apply: allfv_trivial=> - [|?]. }
  move: H'M'1 => /measure_app_lam' ->.
  rewrite !measure_ren.
  exists ns. split; last done.
  rewrite !app_assoc. apply: Permutation_app_tail.
  by rewrite -!app_assoc.
+ move=> t' M'1 M'2 {}n m.
  rewrite !measure_lam => Hn Hn1s H1ns H2ns.
  have [] := has_var_zero_dec M'1.
  { move=> HM'1.
    have : has_var_zero M'2.
    { move: Hn => /inner_stepG_stepG H HM'2.
      apply: HM'1. move: H HM'2 => /allfv_stepG'. by apply. }
    move: HM'1 => /measure_app_lam -> /measure_app_lam ->.
    exists (n1s ++ [(size_sty t', 0)] ++ measure M3). split.
    { rewrite !app_assoc. apply: Permutation_app_tail.
      apply: Permutation_trans; first by apply: Permutation_app_comm.
      by apply: Permutation_app. }
    exists (n2s ++ [(size_sty t', 0)] ++ measure M3). split; first by apply: Forall2_app.
    exists ns. split; last done.
    rewrite !app_assoc. apply: Permutation_app_tail.
    rewrite -app_assoc. apply: Permutation_trans; last by apply: Permutation_app_comm.
    rewrite !app_assoc. by apply: Permutation_app. }
  move=> HM'1.
  have : not (has_var_zero M'2).
  { move: Hn => /inner_stepG_stepG H HM'2.
    apply: HM'1=> HM'1. apply: HM'2. move: H HM'1 => /allfv_stepG. by apply. }
  move: HM'1 => /measure_app_lam' -> /measure_app_lam' ->.
  eexists (n1s ++ _ ++ measure M3). split.
  { rewrite !app_assoc. apply: Permutation_app_tail.
    apply: Permutation_trans; first by apply: Permutation_app_comm.
    by apply: Permutation_app. }
  move: (map inc_depth (measure M'2)) H1ns=> ??.
  move E: (n, m) Hn=> nm Hn.
  eexists (n2s ++ _ ++ measure M3). split; first by apply: Forall2_app.
  case: Hn E H2ns.
  * move=> ? > ?? [-> ->] ?.
    eexists ([_] ++ ns). split.
    { rewrite !app_assoc. apply: Permutation_app_tail.
      rewrite -!app_assoc app_nil_r. by apply: Permutation_app. }
    apply /Forall_app. split; last done.
    constructor; last done.
    move=> /=. lia.
  * move=> *. exists ns. split; last done.
    rewrite !app_assoc. apply: Permutation_app_tail.
    apply: Permutation_trans; last by apply: Permutation_app_comm.
    by apply: Permutation_app.
  * move=> *. exists ns. split; last done.
    rewrite !app_assoc. apply: Permutation_app_tail.
    apply: Permutation_trans; last by apply: Permutation_app_comm.
    by apply: Permutation_app.
  * move=> *. exists ns. split; last done.
    rewrite !app_assoc. apply: Permutation_app_tail.
    apply: Permutation_trans; last by apply: Permutation_app_comm.
    by apply: Permutation_app.
+ move=> > Hn Hn1s H1ns H2ns.
  exists (n1s ++ measure M3). split.
  { rewrite !measure_app_app app_assoc. by apply: Permutation_app_tail. }
  exists (n2s ++ measure M3). split; first by apply: Forall2_app.
```

```
      exists ns. split; last done.
      rewrite !measure_app_app app_assoc. by apply: Permutation_app_tail.
    + move=> > Hn Hn1s H1ns H2ns.
      exists (n1s ++ measure M3). split.
      { rewrite !measure_app_app app_assoc. by apply: Permutation_app_tail. }
      exists (n2s ++ measure M3). split; first by apply: Forall2_app.
      exists ns. split; last done.
      rewrite !measure_app_app app_assoc. by apply: Permutation_app_tail.
  - move=> > ? IH >.
    move=> /stlcE [>] ?.
    move=> /IH [n1s] [Hn1s] [n2s] [Hn1sn2s] [ns] [Hn2s Hns].
    rewrite /measure -/measure. eexists (n1s ++ _). split.
    { rewrite !app_assoc. apply: Permutation_trans; first by apply: Permutation_app_comm.
      by apply: Permutation_app. }
    eexists (n2s ++ _ ++ measure _). split; first by apply: Forall2_app.
    exists ns. split; last done.
    rewrite !app_assoc.
    apply: Permutation_trans; last by apply: Permutation_app_comm.
    rewrite !app_assoc. apply: Permutation_app; last done.
    by apply: Permutation_app.
Qed.

(* decreasing measure relation *)
Inductive measure_lt (mN mM : list (nat*nat)) : Prop :=
  | measure_lt_intro n n1s ns n2s :
    Permutation ([n] ++ n1s) mM ->
    Permutation (ns ++ n2s) mN ->
    Forall (fun '(n', m') => n' < fst n \/ n' = fst n /\ m' < snd n) ns ->
    Forall2 (fun '(n1, m1) '(n2, m2) => n1 = n2 /\ m2 <= m1) n1s n2s ->
    measure_lt mN mM.

Definition lt2 := @slexprod nat nat (Nat.lt) (Nat.lt).

Lemma lt2_spec x y : lt2 x y <-> (fst x < fst y \/ fst x = fst y /\ snd x < snd y).
Proof.
  split.
  - case=> /=; lia.
  - move: x y => [a b] [a' b'] /=.
    move=> [?|[<- ?]].
    + by apply: left_slex.
    + by apply: right_slex.
Qed.

Lemma lt2_wf : forall x, Acc lt2 x.
Proof.
  move=> [a]. elim /(Nat.measure_induction _ id) : a.
  rewrite /id=> a IHa b. elim /(Nat.measure_induction _ id) : b.
  rewrite /id=> b IHb. constructor=> - [a' b'] /lt2_spec /= [].
  - by move=> /IHa.
  - by move=> [->] /IHb.
Qed.

Lemma clos_refl_trans_list_lt_intro l l' :
  Forall2 (fun '(n1, m1) '(n2, m2) => n1 = n2 /\ m2 <= m1) l' l ->
  clos_refl_trans (list (nat * nat)) (list_lt lt2) l l'.
Proof.
  elim; first by constructor.
  move=> [a1 b1] [a2 b2] /= > [<-] *. apply: rt_trans.
  + apply: (clos_refl_trans_list_lt_app_r _ [_]). by eassumption.
  + apply: (clos_refl_trans_list_lt_app_l _ [_] [_]).
    have [<-|?] : b1 = b2 \/ b2 < b1 by lia.
    { by constructor. }
    apply: rt_step.
    rewrite -(app_nil_r [(a1, b2)]) -(app_nil_r [(a1, b1)]).
    apply: (list_lt_intro _ _ [_] nil nil).
    constructor; last done.
    apply /lt2_spec=> /=. lia.
Qed.

(* measure_lt is well-founded *)
Theorem measure_lt_wf : well_founded measure_lt.
Proof.
  apply: (wf_clos_trans_inverse_rel _ (list_lt lt2) (@Permutation (nat * nat))).
  - apply: list_lt_wf. apply: lt2_wf.
  - move=> l. by exists l.
  - move=> l1 l2 l'2 [] n n1s ns n2s Hl2 Hl1 Hns Hn12s Hl2l'2.
    move: Hl2 Hl2l'2 Hl1 => /Permutation_trans /[apply].
    have : In n ([n] ++ n1s).
    { apply /in_app_iff. left. by left. }
    move=> /Permutation_in + /[dup] => /[apply] /in_split [l'3] [l'4] ->.
    move=> /(@Permutation_app_inv (nat*nat) nil) Hn1s Hn2s.
    move: Hn12s Hn1s => /Forall2_permutation_app /[apply].
    move=> [l3] [l4] [?] [H3 H4].
    exists (l3 ++ ns ++ l4). split.
    + apply: Permutation_trans; first by (apply: Permutation_sym; eassumption).
      by apply: (Permutation_app_middle _ nil).
    + apply: (@clos_rt_t _ _ _ (l'3 ++ ns ++ l'4)).
```

```
      { apply: clos_refl_trans_list_lt_intro.
        apply Forall2_app; first done.
        apply Forall2_app; last done.
        elim: (ns); first done.
        move=> [??] ??. constructor; last done. lia. }
      apply: t_step. constructor.
      apply: Forall_impl Hns.
      move: n=> - [??] [??] /= ?. apply /lt2_spec=> /=. lia.
Qed.

Lemma measure_spec Gamma M t : measure M <> [] -> stlc Gamma M t -> exists N y, inner_stepI y M N \/
     inner_stepG y M N.
Proof.
  elim /(Nat.measure_induction _ size_tm): M Gamma t.
  move=> [?|M1 M2|t M] IH.
  - done.
  - move=> ?? + /stlcE [>] ++.
    have [|] : measure M2 <> [] \/ measure M2 = [].
    { by move: (measure M2) => [|??] /=; [right|left]. }
    { move=> /IH {}IH _ _ /IH - [].
      { move=> /=. lia. }
      move=> ? [?] [|] ?.
      * do 2 eexists. left. apply: inner_stepIAppR. by eassumption.
      * do 2 eexists. right. apply: inner_stepGAppR. by eassumption. }
    have [|] : measure M1 <> [] \/ measure M1 = [].
    { by move: (measure M1) => [|??] /=; [right|left]. }
    { move=> /IH {}IH _ _ /IH - [].
      { move=> /=. lia. }
      move=> ? [?] [|] ?.
      * do 2 eexists. left. apply: inner_stepIAppL. by eassumption.
      * do 2 eexists. right. apply: inner_stepGAppL. by eassumption. }
    rewrite /measure -/measure.
    move=> /[dup] HM1 -> /[dup] HM2 ->. rewrite !app_nil_r.
    move: M1 HM1 IH=> [?|t' M'1 M'2|t' M'] IH.
    + done.
    + done.
    + case: (has_var_zero_dec M')=> /=.
      { move=> ??? /stlcE [>] ? [??] ??. subst.
        do 2 eexists. left. by apply: inner_stepIRed. }
      move: M' IH=> [?|???|??]; [done..|].
      move=> ??? _ /stlcE [>] /stlcE [>] ? ??. subst.
      move=> [??] ??. subst.
      do 2 eexists. right. by apply: inner_stepGRed.
  - move=> >. rewrite measure_lam.
    move=> /map_neq_nil /IH {}IH.
    move=> /stlcE [>] /IH - [].
    { move=> /=. lia. }
    move=> ? [[??]] [|] *; subst.
    + do 2 eexists. left. constructor. by eassumption.
    + do 2 eexists. right. constructor. by eassumption.
Qed.

(* any simply typed term (I, gamma)-reduces to a term with an empty measure *)
Lemma stepIG_normalization Gamma M t : stlc Gamma M t -> exists N, clos_refl_trans _ (fun P Q => stepI P Q
     \/ stepG P Q) M N /\ measure N = [] /\ stlc Gamma N t.
Proof.
  have := measure_lt_wf (measure M).
  move E: (measure M)=> n H. elim: H M E.
  move=> l _ IH M ?. subst l.
  move E: (measure M)=> [|x ?].
  { move=> ?. exists M. by split; first by apply: rt_refl. }
  have: measure M <> [] by congruence.
  move=> /measure_spec + /[dup] => /[apply] - [N] [y] [|].
  - move=> /[dup] /inner_stepI_stepI /[dup] HMN /stepI_reduction H + /[dup] + /H.
    move=> /measure_decrease_inner_stepI /[apply].
    move=> [?] [?] [?] [?] [?] [??].
    have : measure_lt (measure N) (measure M).
    { econstructor; by eassumption. }
    move=> /IH => /(_ _ eq_refl) /[apply].
    move=> [N'] [??]. exists N'. split; last done.
    apply: rt_trans; last by eassumption.
    apply: rt_step. by left.
  - move=> /[dup] /inner_stepG_stepG /[dup] HMN /stepG_reduction H + /[dup] + /H.
    move=> /measure_decrease_inner_stepG /[apply].
    move=> [?] [?] [?] [?] [?] [??].
    have : measure_lt (measure N) (measure M).
    { econstructor; by eassumption. }
    move=> /IH => /(_ _ eq_refl) /[apply].
    move=> [N'] [??]. exists N'. split; last done.
    apply: rt_trans; last by eassumption.
    apply: rt_step. by right.
Qed.

(* count the number of redexes *)
Fixpoint redex_count (M : tm) : nat :=
  match M with
  | var _ => 0
```

```
  | lam _ M => redex_count M
  | app M N =>
    match M with
    | var _ => 0
    | lam _ _ => 1 + redex_count M
    | app _ _ => redex_count M
    end + redex_count N
  end.

Lemma redex_count_zero_spec M : redex_count M = 0 -> nf M.
Proof.
  elim: M.
  - move=> *. by do 2 constructor.
  - move=> /= [?|??|??] IH1 ? IH2.
    + move=> /IH2 ?. do 2 constructor; last done.
      by constructor.
    + move=> /Nat.eq_add_0 [/IH1] {}IH1 /IH2 {}IH2.
      do 2 constructor; last done.
      by inversion IH1.
    + done.
  - move=> > /= /[apply] ?. by constructor.
Qed.

Lemma measure_app_nil M N : measure (app M N) = [] -> measure M = [] /\ measure N = [].
Proof.
  by rewrite /measure -/measure=> /app_eq_nil [] ? /app_eq_nil.
Qed.

Lemma redex_count_spec Gamma M t : measure M = [] -> redex_count M <> 0 -> stlc Gamma M t -> exists N, stepK
    M N.
  elim /(Nat.measure_induction _ size_tm): M Gamma t.
  move=> [x|M1 M2|t M] IH.
  - done.
  - move=> Gamma s.
    move: M1 IH=> [].
    + move=> x IH /=. rewrite measure_app_var.
      move=> /IH /[apply] {}IH.
      move=> /stlcE [>] ? /IH - [].
      { move=> /=. lia. }
      move=> *. eexists. constructor; last by eassumption.
      by constructor.
    + move=> M'1 M'2 IH. rewrite measure_app_app.
      change (redex_count (app (app M'1 M'2) M2)) with ((redex_count (app M'1 M'2)) + (redex_count M2)).
      move=> /app_eq_nil [] /IH {}IH1 /IH {}IH2 ?.
      have [|] : redex_count (app M'1 M'2) <> 0 \/ (redex_count (app M'1 M'2) = 0 /\ redex_count M2 <> 0) by
      lia.
      * move=> /IH1 {}IH1.
        move=> /stlcE [>] /[dup] /stlcE [>] ??.
        move=> /IH1 - [].
        { move=> /=. lia. }
        move=> *. subst. eexists. apply: stepKAAppL. by eassumption.
      * move=> [/redex_count_zero_spec] H'.
        move=> /IH2 {}IH2 /stlcE [>] _ /IH2 - [].
        { move=> /=. lia. }
        move=> *. eexists. apply: stepKNAppR; last by eassumption.
        inversion H'. subst.
        elim: H; by eauto using hf.
    + move=> t' M' IH.
      have [|HM'] := has_var_zero_dec M'.
      { by move=> /measure_app_lam ->. }
      move: IH.
      have -> : M' = ren S (ren Nat.pred M').
      { rewrite ren_ren -[LHS]ren_id.
        apply: ext_allfv_ren.
        have [] := allfv_dec (fun x : nat => id x = funcomp S Nat.pred x) M'.
        - rewrite /id. by move=> [|?] /=; [right|left].
        - done.
        - move=> H. exfalso. apply: HM'=> HM'. apply: H.
          by apply: allfv_impl HM'=> - [|?] /=. }
      move=> IH H1M2 H2M2 /stlcE [>] /stlcE [>] ? [??] ? H3M2. subst.
      have [|] : (redex_count M2 <> 0) \/ (redex_count M2 = 0) by lia.
      { move: H1M2=> /measure_app_nil [_].
        move: H3M2=> /IH /[apply] /[apply] - [].
        { move=> /=. lia. }
        move=> *. eexists. apply: stepKLAppR. by eassumption. }
      move=> /redex_count_zero_spec /stepKRed H.
      eexists. by apply: H.
  - rewrite measure_lam /=.
    move=> > /map_eq_nil /IH /[apply] {}IH.
    move=> /stlcE [>] /IH - [].
    { move=> /=. lia. }
    move=> *. eexists. constructor. by eassumption.
Qed.

Lemma redex_count_ren xi M : redex_count (ren xi M) = redex_count M.
Proof.
  elim: M xi.
```

```
  - done.
  - move=> M IH1 ? IH2 > /=.
    rewrite IH1 IH2. congr Nat.add.
    by case: (M).
  - move=> > IH >. by apply: IH.
Qed.

Lemma redex_count_stepK M N : stepK M N -> measure M = [] -> measure N = [] /\ redex_count N < redex_count M
    .
Proof.
  elim.
  - move=> > /= ?.
    move=> /[dup] /measure_app_nil [].
    rewrite measure_lam measure_ren redex_count_ren=> /map_eq_nil *.
    by split; [|lia].
  - move=> >. rewrite !measure_lam /=.
    by move=> ? IH /map_eq_nil /IH [-> ?].
  - move=> > ?? IH /[dup] /measure_app_nil [_] /[dup] E /IH [{}IH ?] H. split.
    + move: H. by rewrite /measure -/measure E IH !app_nil_r.
    + move=> /=. lia.
  - move=> > ? IH /[dup] /measure_app_nil [_] /[dup] E /IH [{}IH ?] H. split.
    + move: H. by rewrite /measure -/measure E IH !app_nil_r.
    + move=> /=. lia.
  - move=> M1 M2 M3 {}N H0M3 IH /[dup] /measure_app_nil [] /[dup] /IH {IH} [H1M3 H2M3] ? HN HM''.
    move E: (app M1 M2) H0M3=> M' H.
    case: H E H1M3 H2M3.
    + move=> ? M'' ?? [] ???. subst.
      move: HM''. rewrite measure_app_app measure_app_lam' ?measure_ren.
      { apply. apply: allfv_ren. by apply: allfv_trivial. }
      move=> HM'' H2M3. split; first last.
      { move: H2M3 => /=. rewrite !redex_count_ren.
        by case: (M'') HM''; [lia..|]. }
      rewrite /measure -/measure.
      move: (M'') HM''=> [] /=; last done.
      * by move=> _ /app_eq_nil [].
      * by move=> > /app_eq_nil [] /app_eq_nil [] /map_eq_nil ->.
    + done.
    + move=> > ?? [] ??. subst. rewrite !measure_app_app HN.
      move=> -> H. split; first done.
      move: H => /=. lia.
    + move=> > ? [] ??. subst. rewrite !measure_app_app HN.
      move=> -> H. split; first done.
      move: H => /=. lia.
    + move=> > ? [] ??. subst. rewrite !measure_app_app HN.
      move=> -> H. split; first done.
      move: H => /=. lia.
Qed.

(* if measure is empty, then the term K-reduces to a normal form *)
Lemma stepK_normalization Gamma M t : stlc Gamma M t -> measure M = [] ->
  exists N, clos_refl_trans _ stepK M N /\ redex_count N = 0 /\ stlc Gamma N t.
Proof.
  elim /(Nat.measure_induction _ redex_count): M.
  move=> M IH.
  move E: (redex_count M) IH=> [|?].
  { move=> _ ??. exists M. by split; first by apply: rt_refl. }
  move=> IH.
  have /redex_count_spec : redex_count M <> 0 by congruence.
  move=> H /[dup] /H.
  move=> {}H + /[dup] /H - [N HMN].
  move: (HMN)=> /stepK_reduction {}H /[dup] /H.
  move: (HMN)=> /redex_count_stepK {}H + HM /H [].
  rewrite E=> /IH /[apply] /[apply] => - [N'] [??].
  exists N'. split; last done.
  apply: rt_trans; last by eassumption.
  by apply: rt_step.
Qed.

Theorem stlc_nf M Gamma t : stlc Gamma M t -> exists N, steps M N /\ nf N.
Proof.
  move=> /stepIG_normalization [M'] [HMM'] [].
  move=> /stepK_normalization /[apply] - [N] [HM'N] [/redex_count_zero_spec ?] ?.
  exists N. split; [|done].
  apply: (rt_trans _ _ _ M').
  - elim: HMM'=> *.
    + apply: rt_step. rewrite /step. by tauto.
    + by apply: rt_refl.
    + apply: rt_trans; by eassumption.
  - elim: HM'N=> *.
    + apply: rt_step. rewrite /step. by tauto.
    + by apply: rt_refl.
    + apply: rt_trans; by eassumption.
Qed.
```

Original author formal source: nitlc.v

```
(*
  definitions for the non-idempotently typed lambda-calculus
  "nitlc Gamma M A" means that in the type environment Gamma the term M is assigned the type A
*)

Require Import List Permutation.
Import ListNotations.
From NonIdempotent Require Import stlc.

(* non-itempotent intersection types *)
Inductive nity : Type :=
  | niatom (x : nat) : nity
  | niarr (u : list nity) (A : nity) : nity.

(* type environment *)
Notation environment := (list (list nity)).

(* ssim s A means non-idempotent type A collapses to simple type s *)
Inductive ssim : sty -> nity -> Prop :=
  | ssim_atom x : ssim (satom x) (niatom x)
  | ssim_arr s t A u B : Forall (ssim s) (cons A u) -> ssim t B -> ssim (sarr s t) (niarr (cons A u) B).

(* ssim lifted to type environments *)
Notation env_ssim Gamma Delta :=
  (forall x, match nth_error Gamma x with Some s => Forall (ssim s) (nth x Delta []) | None => True end).

(* environment permutation *)
Notation sty_permutation Gammas Deltas :=
  (forall x, @Permutation nity
    (concat (map (fun Gamma => nth x Gamma nil) Gammas))
    (concat (map (fun Delta => nth x Delta nil) Deltas))).

(* non-idempotently typed lambda-calculus *)
(* nitlc Gamma M A means that in the type environment Gamma the term M is assigned the type A *)
Inductive nitlc (Gamma : environment) : tm -> nity -> Prop :=
  | nitlc_var t x A :
      ssim t A ->
      In A (nth x Gamma nil) ->
      nitlc Gamma (var x) A
  | nitlc_app Gamma' Deltas M N u A :
      sty_permutation [Gamma] (Gamma' :: Deltas) ->
      nitlc Gamma' M (niarr u A) ->
      Forall2 (fun B Delta => nitlc Delta N B) u Deltas ->
      nitlc Gamma (app M N) A
  | nitlc_lam t M u A :
      ssim t (niarr u A) ->
      nitlc (cons u Gamma) M A ->
      nitlc Gamma (lam t M) (niarr u A).
```

Original author formal source: nitlc_facts.v

```
(*
  facts about the non-idempotently typed lambda-calculus
*)

Require Import PeanoNat Lia List Relations Permutation.
Import ListNotations.

From NonIdempotent Require Import facts stlc stlc_facts nitlc.
Require Import ssreflect ssrbool ssrfun.

#[local] Arguments nth_error_split {A l n a}.
#[local] Arguments Permutation_in {A l l' x}.
#[local] Arguments Permutation_nil_cons {A l x}.
#[local] Arguments in_split {A x l}.


Lemma nitlcE Gamma M A : nitlc Gamma M A ->
  match M with
  | var x => (exists t, ssim t A) /\ In A (nth x Gamma nil)
  | app M N => exists Gamma' Deltas u,
      sty_permutation [Gamma] (Gamma' :: Deltas) /\
      nitlc Gamma' M (niarr u A) /\
      Forall2 (fun B Delta => nitlc Delta N B) u Deltas
  | lam t M => exists u B,
      A = niarr u B /\
      ssim t (niarr u B) /\
      nitlc (cons u Gamma) M B
  end.
Proof.
  case; by eauto 10.
Qed.

Notation env_ssim Gamma Delta :=
  (forall x, match nth_error Gamma x with Some s => Forall (ssim s) (nth x Delta []) | None => True end).

Lemma ssimE t A : ssim t A ->
  match t with
  | satom x => A = niatom x
  | sarr s t => exists C u B, A = niarr (cons C u) B /\ Forall (ssim s) (cons C u) /\ ssim t B
  end.
Proof.
  case.
  + done.
  + move=> *.
    do 5 econstructor; eassumption.
Qed.

Lemma nitlc_ssim Gamma M A : nitlc Gamma M A -> exists t, ssim t A.
Proof.
  elim.
  - move=> *. eexists. by eassumption.
  - move=> > ?? [t] /ssimE.
    case: t => [?|??]; first done.
    move=> [?] [?] [?] [[]] ? -> [??] ?.
    eexists. by eassumption.
  - move=> *. eexists. by eassumption.
Qed.

(* intermediate induction principle, not including additional constraints *)
Lemma nitlc_ind' (P : environment -> tm -> nity -> Prop) :
  (forall Gamma t x A,
    ssim t A ->
    In A (nth x Gamma nil) ->
    P Gamma (var x) A) ->
  (forall Gamma Deltas Gamma' t M N u A,
    Deltas <> [] ->
    ssim t A ->
    sty_permutation [Gamma] (Gamma' :: Deltas) ->
    nitlc Gamma' M (niarr u A) ->
    P Gamma' M (niarr u A) ->
    Forall2 (fun (B : nity) (Delta : environment) => nitlc Delta N B) u Deltas ->
    Forall2 (fun (B : nity) (Delta : environment) => P Delta N B) u Deltas ->
    P Gamma (app M N) A) ->
  (forall Gamma t M u B,
    ssim t (niarr u B) ->
    nitlc (u :: Gamma) M B ->
    P (u :: Gamma) M B ->
    P Gamma (lam t M) (niarr u B)) ->
  forall Gamma M A, nitlc Gamma M A -> P Gamma M A.
Proof.
  move=> IH1 IH2 IH3 + M. elim: M.
  - move=> > /nitlcE [] [?]. by apply: IH1.
  - move=> M IHM N IHN Gamma A.
    move=> /[dup] /nitlc_ssim [t ?].
```

```
    move=> /nitlcE [Gamma'] [Deltas] [u] [?] [HM HN].
    apply: IH2; try eassumption.
    + move: HM HN => /nitlc_ssim [?] H. inversion H. subst.
      move=> /Forall2_length. by case: (Deltas).
    + by apply: IHM.
    + apply: Forall2_impl HN => *. by apply: IHN.
  - move=> t M IHM Gamma A /nitlcE.
    move=> [?] [?] [?] [??]. subst A.
    apply: IH3; [assumption..|].
    by apply: IHM.
Qed.

Lemma sty_permutation_mergeL Gamma' Gamma Delta Gammas Deltas :
  sty_permutation (Gamma :: Gammas) (Delta :: Deltas) ->
  sty_permutation ((merge Gamma' Gamma) :: Gammas) ((merge Gamma' Delta) :: Deltas).
Proof.
  move=> H x.
  rewrite /= !nth_merge -!app_assoc.
  by apply: Permutation_app; [|apply: H].
Qed.

Theorem weakening Gamma' Gamma M A :
  nitlc Gamma M A -> nitlc (merge Gamma' Gamma) M A.
Proof.
  move=> H. elim /nitlc_ind': H Gamma'.
  - move=> *. apply: nitlc_var; [by eassumption|]. by apply: In_nth_merge_r.
  - move=> {}Gamma Deltas Gamma' t {}M N u {}A.
    move=> _ Hta HGamma IHM IH'M IHN IH'N Gamma''.
    apply: (nitlc_app _ (merge Gamma'' Gamma') Deltas).
    + by apply (sty_permutation_mergeL _ _ _ nil Deltas).
    + apply: IH'M.
    + by apply: Forall2_impl IH'N => > /(_ nil).
  - move=> {}Gamma t {}M u B ? IH'M IHM Gamma'.
    constructor; [assumption|].
    by apply: (IHM ([] :: Gamma')).
Qed.

Lemma sty_permutation_trans Gammas1 Gammas2 Gammas3 :
  sty_permutation Gammas1 Gammas2 ->
  sty_permutation Gammas2 Gammas3 ->
  sty_permutation Gammas1 Gammas3.
Proof.
  move=> H1 H2 x. move: (H1 x) (H2 x). apply: Permutation_trans.
Qed.

Lemma sty_permutation_sym Gammas1 Gammas2 :
  sty_permutation Gammas1 Gammas2 ->
  sty_permutation Gammas2 Gammas1.
Proof.
  move=> H x. by apply: Permutation_sym.
Qed.

Lemma sty_permutation_app Gammas1 Gammas2 Deltas1 Deltas2 :
  sty_permutation Gammas1 Deltas1 ->
  sty_permutation Gammas2 Deltas2 ->
  sty_permutation (Gammas1 ++ Gammas2) (Deltas1 ++ Deltas2).
Proof.
  move=> H1 H2 x. rewrite !map_app !concat_app. by apply: Permutation_app.
Qed.

Lemma sty_permutation_app_comm Gammas1 Gammas2 :
  sty_permutation (Gammas1 ++ Gammas2) (Gammas2 ++ Gammas1).
Proof.
  move=> x. rewrite !map_app !concat_app. by apply: Permutation_app_comm.
Qed.

(* actually can be a special case of weakening with <= instead of permutation *)
Theorem permutation Gamma1 Gamma2 M A : sty_permutation [Gamma1] [Gamma2] -> nitlc Gamma1 M A -> nitlc
    Gamma2 M A.
Proof.
  move=> + H. elim /nitlc_ind': H Gamma2.
  - move=> {}Gamma1 t x {}A ?? Gamma2 H. econstructor; [by eassumption|].
    move: H=> /(_ x) /Permutation_in=> /(_ A) /=.
    rewrite !app_nil_r. by apply.
  - move=> {}Gamma1 Deltas Gamma' t {}M N u {}A ??? IH1M IH2M IH1N IH2N.
    move=> Gamma2 /sty_permutation_sym ?.
    econstructor; [ |by eassumption..].
    apply: sty_permutation_trans; by eassumption.
  - move=> {}Gamma t {}M u B ?? IH Gamma2 H.
    constructor; [done|].
    by apply: IH=> - [|x] /=; [|apply: H].
Qed.

Lemma sty_permutation_skipn n Gamma Deltas :
  sty_permutation [Gamma] Deltas ->
  sty_permutation [skipn n Gamma] (map (skipn n) Deltas).
Proof.
```

```
  move=> H x. move: (H (n + x))=> /=.
  rewrite nth_skipn map_map.
  congr Permutation. congr concat.
  apply: map_ext=> ?. by rewrite nth_skipn.
Qed.

Lemma ssim_inj A s t : ssim s A -> ssim t A -> s = t.
Proof.
  elim: s t A.
  - move=> ? [?|??] ?.
    + by move=> /ssimE -> /ssimE [->].
    + by move=> /ssimE -> /ssimE [?] [?] [?] [].
  - move=> s1 IH1 s2 IH2 [?|t1 t2] ?.
    + by move=> /ssimE [?] [?] [?] [->] ? /ssimE.
    + move=> /ssimE [?] [?] [?] [->] [/Forall_cons_iff [Hs1 ?] Hs2].
      move=> /ssimE [?] [?] [?] [[]] ??? [/Forall_cons_iff [Ht1 ?] Ht2]. subst.
      congr sarr.
      * apply: IH1; by eassumption.
      * apply: IH2; by eassumption.
Qed.

Fixpoint lift_sty (s : sty) : nity :=
  match s with
  | satom x => niatom x
  | sarr s t => niarr (cons (lift_sty s) nil) (lift_sty t)
  end.

Lemma ssim_lift_sty {t} : ssim t (lift_sty t).
Proof.
  elim: t.
  - constructor.
  - move=> s IHs t IHt /=. constructor.
    + by constructor.
    + done.
Qed.

Lemma Forall2_merge Gamma Gamma1 Gamma2 :
  Forall2 (fun s u => Forall (ssim s) u) Gamma Gamma1 ->
  Forall2 (fun s u => Forall (ssim s) u) Gamma Gamma2 ->
  Forall2 (fun s u => Forall (ssim s) u) Gamma (merge Gamma1 Gamma2).
Proof.
  move=> H. elim: H Gamma2.
  - move=> [|??].
    + constructor.
    + by move=> /Forall2E.
  - move=> s u1 {}Gamma {}Gamma1.
    move=> ?? IH [|u2 Gamma2].
    + by move=> /Forall2E.
    + move=> /Forall2E [??] /=.
      constructor.
      * apply /Forall_app.
        by constructor.
      * by apply: IH.
Qed.

Lemma sty_permutation_fold_right_merge Gamma Deltas : sty_permutation [fold_right merge Gamma Deltas] (Gamma
    :: Deltas).
Proof.
  elim: Deltas Gamma.
  - done.
  - move=> Delta Deltas IH Gamma x.
    rewrite /= !app_nil_r nth_merge.
    apply: (Permutation_app_middle _ []).
    have /= := IH Gamma x.
    by rewrite app_nil_r.
Qed.

(* a simply-typed normal form is typable in the uniform non-itempotent type system *)
Theorem stlc_nitlc_nf {Gamma M t} :
  nf M -> stlc Gamma M t ->
  exists Gamma' A', nitlc Gamma' M A' /\ Forall2 (fun s u => Forall (ssim s) u) Gamma Gamma' /\ ssim t A'.
Proof.
  move=> HM.
  move: M HM Gamma t.
  (* a neutral term can be assigned any suitable type *)
  apply: (nf_ind' _ (fun M _ => forall Gamma t, stlc Gamma M t -> forall A', ssim t A' -> exists Gamma',
     nitlc Gamma' M A' /\ Forall2 (fun s u => Forall (ssim s) u) Gamma Gamma')).
  - move=> s M HM IH Gamma t /stlcE [] s' t' /IH + ??. subst.
    move=> [Gamma'] [A'] [H'M] [/Forall2E].
    move: Gamma' H'M => [|u Gamma']; [done|].
    move: u => [|C u] H'M [Hu HGamma] Ht'.
    + exists Gamma', (niarr [lift_sty s'] A').
      do ? apply: conj.
      * constructor; [(do ? constructor); [by apply: ssim_lift_sty|done]|].
        by apply: (weakening [[lift_sty s']]) H'M.
      * done.
      * do ? constructor; [|done].
```

```
        by apply: ssim_lift_sty.
    + exists Gamma', (niarr (C :: u) A').
      constructor; [|constructor].
      * constructor; [by constructor|done].
      * done.
      * by constructor.
  - move=> M HM IH Gamma t /IH /(_ (lift_sty t) ssim_lift_sty).
    move=> [Gamma'] [? ?].
    exists Gamma', (lift_sty t).
    constructor; [|constructor].
    + done.
    + done.
    + apply: ssim_lift_sty.
  - move=> x Gamma t /stlcE Hx A' Ht.
    move: Hx => /nth_error_split [Gamma1] [Gamma2] [-> <-].
    exists ((repeat nil (length Gamma1)) ++ (cons A' nil) :: (repeat nil (length Gamma2))).
    constructor.
    + apply: (nitlc_var); [by eassumption|].
      rewrite app_nth2 repeat_length; [lia|].
      rewrite Nat.sub_diag. by left.
    + elim: Gamma1.
      * constructor; [by constructor|].
        elim: (Gamma2); by constructor.
      * move=> ??? /=.
        constructor; [by constructor|done].
  - move=> M N HM IHM HN IHN Gamma t /stlcE [].
    move=> s /IHM {}IHM + A'3.
    move=> /IHN [Gamma'2] [A'2] [H'N] [H2Gamma HA'2].
    move=> Ht.
    move: (IHM (niarr [A'2] A'3)) => [].
    { by do ? constructor. }
    move=> Gamma'1 [H'M] H1Gamma.
    exists (fold_right merge Gamma'1 [Gamma'2]).
    constructor.
    + apply: (nitlc_app _ Gamma'1 [Gamma'2]); [ |by eassumption|by constructor].
      by apply: sty_permutation_fold_right_merge.
    + by apply: Forall2_merge.
Qed.

Lemma sty_permutation_In_nth Gamma Delta Deltas x A :
  sty_permutation [Gamma] Deltas ->
  In A (nth x Delta []) ->
  In Delta Deltas ->
  In A (nth x Gamma []).
Proof.
  move=> /(_ x) /=. rewrite app_nil_r.
  move=> /Permutation_sym /Permutation_in H ??.
  apply: H. apply /in_concat.
  do ? econstructor; [|by eassumption].
  apply /in_map_iff.
  by do ? econstructor.
Qed.

Lemma sty_permutation_ssim_nth Gamma Deltas x t :
  sty_permutation [Gamma] Deltas ->
  Forall (ssim t) (nth x Gamma []) -> Forall (fun Delta => Forall (ssim t) (nth x Delta [])) Deltas.
Proof.
  move=> /(_ x) /=. rewrite app_nil_r.
  by move=> /Permutation_Forall /[apply] /Forall_concat /Forall_map.
Qed.

Lemma Permutation_remove_elt {X : Type} {x : X} {Delta1 xs Delta2 Gamma xi} f:
  (forall x1 x2, xi x1 = xi x2 -> x1 = x2) ->
  (forall y, Permutation (nth y (Delta1 ++ (x :: xs) :: Delta2) [] ++ f y) (nth (xi y) Gamma [])) ->
  exists Gamma1 : list (list X),
    length Gamma1 = xi (length Delta1) /\
    (exists (ys1 ys2 : list X) (Gamma2 : list (list X)),
      Gamma = Gamma1 ++ (ys1 ++ x :: ys2) :: Gamma2 /\
      (forall y,
        Permutation
          (nth y (Delta1 ++ xs :: Delta2) [] ++ f y)
          (nth (xi y) (Gamma1 ++ (ys1 ++ ys2) :: Gamma2) []))).
Proof.
  move=> Hxi H.
  have : In x (nth (xi (length Delta1)) Gamma []).
  { have := H (length Delta1). rewrite nth_middle /=.
    move=> /(@Permutation_in _ _ _ x). apply. by left. }
  move=> /In_nth [Gamma1] [ys1] [ys2] [Gamma2] [?] HG1.
  subst Gamma. exists Gamma1. split; first done.
  exists ys1, ys2, Gamma2. split; first done.
  move=> y. have /= := H y.
  have [->|Hy] : (y = length Delta1) \/ (y <> length Delta1) by lia.
  - rewrite -HG1 !nth_middle. by apply: (@Permutation_app_inv _ nil).
  - congr Permutation.
    + congr List.app. by apply: nth_middle_neq.
    + apply: nth_middle_neq. move=> ?. apply: Hy. apply: Hxi. lia.
Qed.
```

```
Lemma Permutation_concat_split {X : Type} xi (Deltas : list (list (list X))) Gamma :
  (forall x1 x2, xi x1 = xi x2 -> x1 = x2) ->
  (forall x, Permutation (concat (map ((nth x)^~ []) Deltas)) (nth (xi x) Gamma [])) ->
  exists Delta' Deltas',
    (forall x, Permutation (nth x Gamma []) (concat (map ((nth x)^~ []) (Delta' :: Deltas')))) /\
    Forall2 (fun Delta Delta' => forall x, nth x Delta [] = nth (xi x) Delta' []) Deltas Deltas'.
Proof.
  move=> Hxi.
  elim /(Nat.measure_induction _ (fun Deltas => list_sum (map (fun Delta => 1 + list_sum (map (@length X)
     Delta)) Deltas))) : Deltas Gamma.
  move=> [|Delta Deltas].
  { move=> _ Gamma HG. exists Gamma, nil. split.
    + move=> x /=. by rewrite app_nil_r.
    + done. }
  move=> IH Gamma.
  have [|[Delta1 [x [xs [Delta2 Hx]]]]] : (forall x, nth x Delta [] = []) \/
    exists Delta1 x xs Delta2, Delta = Delta1 ++ (x :: xs) :: Delta2.
  { elim: (Delta).
    - left. by case.
    - move=> [|??] ? [?|].
      + by left=> - [|?] /=.
      + move=> [?] [?] [?] [?] ->.
        right. by eexists (nil :: _), _, _, _.
      + right. by eexists nil, _, _, _.
      + move=> [?] [?] [?] [?] ->.
        right. by eexists ((_ :: _) :: _), _, _, _. }
  - move=> HD /= HDDs.
    have := IH Deltas _ Gamma. case.
    + move=> /=. lia.
    + move=> x. apply: Permutation_trans; [|by apply: HDDs x].
      by rewrite HD.
    + move=> Delta' [Deltas'] [IH1 IH2].
      exists Delta', ([] :: Deltas'). split=> /=.
      * move=> [|x]; by apply: IH1.
      * constructor; [|done].
        move=> x. by move: (xi x) => [|?]; rewrite HD.
  - subst Delta. move=> /= Hx.
    have [Gamma1 [HD1 [ys1 [ys2 [Gamma2 [? HG]]]]]] := Permutation_remove_elt _ Hxi Hx.
    subst Gamma.
    have := IH ((Delta1 ++ xs :: Delta2) :: Deltas) _ (Gamma1 ++ (ys1 ++ ys2) :: Gamma2). case.
    + rewrite /= !map_app !list_sum_app /=. lia.
    + done.
    + move=> Delta'' [[|Delta' Deltas']] [IH1 /Forall2E]; [done|].
      move=> [IH2 ?].
      exists Delta'', ((insert x (xi (length Delta1)) Delta') :: Deltas'). split.
      * move=> y. have /= := IH1 y.
        have [->|Hy] : (y = length Gamma1) \/ (y <> length Gamma1) by lia.
        ** rewrite !nth_middle HD1 nth_insert. by apply: Permutation_elt.
        ** have : y <> xi (length Delta1) by lia.
           move=> /nth_insert_neq ->.
           apply: Permutation_trans.
           by rewrite (nth_middle_neq _ (ys1 ++ ys2)).
      * constructor; [|done].
        move=> y. have := IH2 y.
        have [->|Hy] : (y = length Delta1) \/ (y <> length Delta1) by lia.
        ** rewrite !nth_middle nth_insert. by move=> ->.
        ** have : xi y <> xi (length Delta1).
           { move=> ?. apply: Hy. by apply: Hxi. }
           move=> /nth_insert_neq -> <-.
           by apply: nth_middle_neq.
Qed.

Lemma nitlc_ren Gamma Delta xi A M :
  (forall x1 x2, xi x1 = xi x2 -> x1 = x2) ->
  (forall x, nth x Gamma nil = nth (xi x) Delta nil) ->
  nitlc Gamma M A ->
  nitlc Delta (ren xi M) A.
Proof.
  move=> Hxi H H'. elim /nitlc_ind': H' xi Delta H Hxi.
  - move=> > /= ? ? > H ?. econstructor; [|rewrite -H]; by eassumption.
  - move=> {}Gamma Deltas Gamma' t {}M N u ? ?? HG ? IHM ? IHN xi Gamma'' HMN Hxi.
    have : forall x, Permutation (concat (map ((nth x)^~ []) (Gamma' :: Deltas))) (nth (xi x) Gamma'' []).
    { move=> x. rewrite -HMN. apply /Permutation_sym.
      have := HG x.
      by rewrite /= app_nil_r. }
    move: (Hxi) => /Permutation_concat_split /[apply].
    move=> [G0] [[|Gamma''' Deltas']].
    { by move=> [?] /Forall2_length. }
    move=> [HG''] /= /Forall2E [HG' HD].
    apply: (nitlc_app _ (merge G0 Gamma''') Deltas' _ _ u).
    + move=> x /=.
      by rewrite app_nil_r nth_merge -app_assoc.
    + apply: weakening. by apply: IHM.
    + apply: Forall2_trans; [|by eassumption..].
      move=> > H' /= /H'. by apply.
  - move=> > ?? IH > /= HM Hxi. constructor; first done.
```

```
    apply: IH; [by case|].
    move=> [|x1] [|x2] /=; [done..|].
    by move=> [/Hxi <-].
Qed.

Lemma allfv_make_injective M (xi : nat -> nat) : allfv (fun x1 => allfv (fun x2 => xi x1 = xi x2 -> x1 = x2)
      M) M ->
  exists xi', allfv (fun x => xi x = xi' x) M /\ (forall x1 x2, xi' x1 = xi' x2 -> x1 = x2).
Proof.
  move=> H.
  have [l Hl] := allfv_list M.
  have H' : Forall (fun x1 => Forall (fun x2 => xi x1 = xi x2 -> x1 = x2) l) l.
  { apply /Hl. by apply: allfv_impl H => ? /Hl. }
  suff: exists xi' : nat -> nat,
    Forall (fun x => xi x = xi' x) l /\ (forall x1 x2 : nat, xi' x1 = xi' x2 -> x1 = x2).
  { move=> [xi'] [??]. exists xi'. by split; [apply /Hl|]. }
  clear H Hl.
  have D := In_dec Nat.eq_dec.
  pose k := S (list_sum (map xi l)).
  exists (fun i => if D i l then xi i else k + i). split.
  - apply /Forall_forall => i ?. by case: (D i l).
  - move=> x1 x2. move: (D x1 l) (D x2 l) => [Hx1|Hx1] [Hx2|Hx2] /=.
    + by move: H' Hx1 Hx2 => /Forall_forall /[apply] /Forall_forall /[apply].
    + move: Hx1 => /in_split [?] [?] ->.
      rewrite map_app /= list_sum_app /=. lia.
    + move: Hx2 => /in_split [?] [?] ->.
      rewrite map_app /= list_sum_app /=. lia.
    + lia.
Qed.

Lemma Permutation_allfv_split M (Delta : environment) Deltas Gamma' :
  allfv (fun x => Permutation (concat (map ((nth x)^~ []) (Delta :: Deltas))) (nth x Gamma' [])) M ->
  exists Delta' Deltas',
    (forall x, Permutation (nth x Gamma' []) (concat (map ((nth x)^~ []) (Delta' :: Deltas')))) /\
    (allfv (fun x => nth x Delta [] = nth x Delta' []) M) /\
    Forall2 (fun Delta Delta' => allfv (fun x => nth x Delta [] = nth x Delta' []) M) Deltas Deltas'.
Proof.
  have [l Hl] := allfv_list M.
  have D := In_dec Nat.eq_dec.
  move=> /Hl /Forall_forall HM.
  exists (map (fun i => nth i (if D i l then Delta else Gamma') []) (seq 0 (length Gamma'))).
  exists (map (fun Delta => map (fun i => nth i (if D i l then Delta else []) []) (seq 0 (length Delta)))
     Deltas).
  do ? split.
  - move=> i /=.
    have [|] : i < length Gamma' \/ i >= length Gamma' by lia.
    + move=> /nth_map_seq_lt ->.
      case: (D i l).
      * move=> Hi /=.
        have /Permutation_sym /Permutation_trans /= := HM _ Hi. apply.
        apply: Permutation_eq. congr List.app. congr concat.
        rewrite !map_map. apply: map_ext => Delta'.
        have [|] : i < length Delta' \/ i >= length Delta' by lia.
        ** move=> /nth_map_seq_lt ->. by case: (D i l).
        ** move=> /[dup] ? /nth_map_seq_ge ->.
           by rewrite (nth_overflow Delta').
      * move=> Hi /=. apply: Permutation_eq. apply: eq_trans.
        { apply: eq_sym. by apply: app_nil_r. }
        congr List.app. elim: (Deltas); first done.
        move=> Delta' Deltas' IH /=.
        rewrite -IH app_nil_r.
        have [|] : i < length Delta' \/ i >= length Delta' by lia.
        ** move=> /nth_map_seq_lt ->. by move: (i) (D i l) Hi => [|?] [|] /=.
        ** by move=> /nth_map_seq_ge ->.
    + move=> /[dup] ? /nth_map_seq_ge ->.
      rewrite (nth_overflow Gamma') /=; first done.
      apply: Permutation_eq. case: (D i l) => Hi.
      * have := HM _ Hi.
        rewrite (nth_overflow Gamma') /=; first done.
        move=> H.
        have : Forall (fun Delta' => nth i Delta' [] = []) Deltas.
        { apply /Forall_forall => Delta' HDelta'.
          move: HDelta' H => /in_split [?] [?] ->.
          rewrite map_app concat_app /=.
          case: (nth i Delta' []); first done.
          move=> s ? /=.
          move=> /(Permutation_in) => /(_ s) H. exfalso.
          apply: H. do 2 (apply /in_app_iff; right). by left. }
        elim; first done.
        move=> Delta' Deltas' IH ? /= <-.
        rewrite app_nil_r.
        have [|] : i < length Delta' \/ i >= length Delta' by lia.
        ** move=> /nth_map_seq_lt ->. case: (D i l); last done.
           by rewrite /= IH.
        ** by move=> /nth_map_seq_ge ->.
      * elim: (Deltas); first done.
        move=> Delta' Deltas' /= <-. rewrite app_nil_r.
```

```
        have [|] : i < length Delta' \/ i >= length Delta' by lia.
        ** move=> /nth_map_seq_lt ->. by move: (i) (D i l) Hi => [|?] [|] /=.
        ** by move=> /nth_map_seq_ge ->.
  - apply /Hl. apply /Forall_forall => i Hi.
    have [|] : i < length Gamma' \/ i >= length Gamma' by lia.
    + move=> /nth_map_seq_lt ->. by case: (D i l).
    + move=> /[dup] ? /nth_map_seq_ge ->.
      have := HM _ Hi. rewrite (nth_overflow Gamma') /=; first done.
      case: (nth i Delta []); first done.
      by move=> > /Permutation_sym /Permutation_nil_cons.
  - apply: Forall2_map_r. elim: (Deltas); first done.
    move=> Delta' Deltas' IH. constructor; last done.
    apply /Hl. apply /Forall_forall => i Hi.
    have [|] : i < length Delta' \/ i >= length Delta' by lia.
    + move=> /nth_map_seq_lt ->. by case: (D i l).
    + move=> /[dup] ? /nth_map_seq_ge ->.
      by rewrite (nth_overflow Delta').
Qed.

Lemma nitlc_allfv Gamma Delta A M :
  allfv (fun x => nth x Gamma nil = nth x Delta nil) M ->
  nitlc Gamma M A ->
  nitlc Delta M A.
Proof.
  move=> H H'. elim /nitlc_ind': H' Delta H.
  - move=> > /= ? ? > H. econstructor; [|rewrite -H]; by eassumption.
  - move=> {}Gamma Deltas Gamma' t {}M N u ? ?? HG ? IHM ? IHN Gamma'' HMN.
    have : allfv (fun x => Permutation (concat (map ((nth x)^~ []) (Gamma' :: Deltas))) (nth x Gamma'' []))
     (app M N).
    { apply: allfv_impl HMN => x H.
      have /Permutation_sym := HG x.
      by rewrite /= H app_nil_r. }
    move=> /Permutation_allfv_split.
    move=> [Gamma'''] [Deltas'].
    move=> [HG''] /= [[HM HN]] ?.
    apply: (nitlc_app _ Gamma''' Deltas' _ _ u).
    + move=> x /=.
      by rewrite app_nil_r.
    + apply: IHM. by apply: allfv_impl HM.
    + apply: Forall2_trans; [|by eassumption..].
      by move=> > H' /= [_ /H'].
  - move=> > ?? IH > /= HM. constructor; first done.
    apply: IH. apply: allfv_impl HM. by case.
Qed.

Lemma nitlc_allfv_ren xi Gamma Delta A M :
  allfv (fun x1 => allfv (fun x2 => xi x1 = xi x2 -> x1 = x2) M) M ->
  allfv (fun x => nth x Gamma nil = nth (xi x) Delta nil) M ->
  nitlc Gamma M A ->
  nitlc Delta (ren xi M) A.
Proof.
  move=> /allfv_make_injective [xi'] [H1xi' H2xi'] H.
  have -> : ren xi M = ren xi' M by apply: ext_allfv_ren.
  have {}H : allfv (fun x => nth x Gamma [] = nth (xi' x) Delta []) M.
  { apply: allfv_allfv_impl H. apply: allfv_impl H1xi'.
    by move=> ? ->. }
  clear xi H1xi'.
  have : exists Gamma' Delta',
    allfv (fun x => nth x Gamma nil = nth x Gamma' nil) M /\
    allfv (fun x => nth x Delta' nil = nth x Delta nil) (ren xi' M) /\
    (forall x, nth x Gamma' nil = nth (xi' x) Delta' nil).
  { have [l Hl] := allfv_list M.
    suff: exists (Gamma' Delta' : environment),
      Forall (fun x =>
        (nth x Gamma [] = nth x Gamma' []) /\
        (nth (xi' x) Delta' [] = nth (xi' x) Delta [])) l /\
      (forall x , nth x Gamma' [] = nth (xi' x) Delta' []).
    { move=> [Gamma'] [Delta'] [/Hl HM] ?.
      exists Gamma', Delta'. do ? split.
      - apply: allfv_impl HM. tauto.
      - apply: allfv_ren. apply: allfv_impl HM. tauto.
      - done. }
    move: H => /Hl. clear -H2xi'.
    elim: l Gamma Delta.
    - move=> *. exists nil, nil. split; first done.
      move=> x. by move: x (xi' x) => [|?] [|?].
    - move=> i l IH Gamma Delta.
      move=> /Forall_cons_iff [?] /IH.
      move=> [Gamma'] [Delta'] [H'l ?].
      exists (replace (nth i Gamma []) i Gamma').
      exists (replace (nth (xi' i) Delta []) (xi' i) Delta'). do ? split.
      + constructor.
        * by rewrite !nth_replace.
        * apply: Forall_impl H'l => j [?] ?. split.
          ** case: (Nat.eq_dec j i) => /=.
             *** move=> <-. by rewrite nth_replace.
             *** by move=> /nth_replace_neq ->.
```

```
          ** case: (Nat.eq_dec (xi' j) (xi' i)) => /=.
             *** move=> <-. by rewrite nth_replace.
             *** by move=> /nth_replace_neq ->.
      + move=> j. case: (Nat.eq_dec j i) => /=.
        * move=> ->. by rewrite !nth_replace.
        * move=> /[dup] Hji /nth_replace_neq ->.
          have : xi' j <>  xi' i.
          { move=> ?. apply: Hji. by apply: H2xi'. }
          by move=> /nth_replace_neq ->. }
  move=> [Gamma'] [Delta'] [HG'] [HD'] ?.
  move: HG' => /nitlc_allfv /[apply].
  move: (H2xi') => /nitlc_ren /[apply] H'.
  move: HD' => /nitlc_allfv. apply.
  by apply: H'.
Qed.
```

Original author formal source: nitlc_typ.v

```
(*
  main result: theorem "nitlc_type_inference" states that
  if a term M is assigned the simple type s in the simple type environemnt Gamma0,
  then there is a uniform non-idempotent type environment Gamma (collapsing to Gamma0)
  and a uniform non-idempotent type A (collapsing to s) such that
  M is assigned the type A in the type environment Gamma

  supplementary result: theorem "nitlc_stlc" states that
  if a term M is assigned the uniform non-idempotent type A in the uniform non-idempotent type environemnt
     Gamma
  such that Gamma collapses to a simple type environment Gamma0,
  then A collapses to a simple type t such that
  the term M is assigned the type t in the environemnt Gamma0.
*)

Require Import PeanoNat Lia List Relations Permutation.
Import ListNotations.

From NonIdempotent Require Import facts stlc stlc_facts stlc_nf nitlc nitlc_facts.
Require Import ssreflect ssrfun.

Arguments nth_error_split {A l n a}.
Arguments in_split {A x}.
Arguments in_combine_r {A B l l' x y}.
Arguments In_nth_error {A l x}.
Arguments map_nth_error {A B f n l d}.

#[local] Notation environment := (list (list nity)).

(* a term is simply typable, if it is typable in the uniform non-itempotent type system *)
Theorem nitlc_stlc Gamma0 M Gamma A : nitlc Gamma M A ->
  env_ssim Gamma0 Gamma ->
  allfv (fun x => nth_error Gamma0 x <> None) M ->
  exists t, stlc Gamma0 M t /\ ssim t A.
Proof.
  move=> H. elim /nitlc_ind': H Gamma0.
  - move=> Gamma' t x A' Ht Hx Gamma0 H' H''.
    exists t. constructor; [|done].
    constructor. move: (H' x) H'' => /=.
    move E: (nth_error Gamma0 x) => [?|]; last done.
    move=> /Forall_forall => /(_ _ Hx) ? _.
    congr Some. apply: ssim_inj; by eassumption.
  - move=> {}Gamma' [|Delta Deltas]; first done.
    move=> Gamma'' t' {}M N [|B' u'] {}A' _.
    { by move=> _ _ _ _ /Forall2_length. }
    move=> H HGamma' ? IHM ? /Forall2E [] IH1N IH2N Gamma0 /= H' [H'M H'N].
    exists t'. split; [|done].
    have : env_ssim Gamma0 Gamma''.
    { move=> x. have := H' x. move E: (nth_error _ _) => [?|]; last done.
      by move: HGamma' => /sty_permutation_ssim_nth /[apply] /Forall_cons_iff []. }
    move: H'M => /IHM /[apply].
    move=> [] [?|??] [] H''M /ssimE; first done.
    move=> [?] [?] [?] [[]] ??? [/Forall_cons_iff [??] ?].
    have : env_ssim Gamma0 Delta.
    { move=> x. have := H' x. move E: (nth_error _ _) => [?|]; last done.
      by move: HGamma' => /sty_permutation_ssim_nth /[apply] /Forall_cons_iff [?] /Forall_cons_iff []. }
    move: (H'N) => /IH1N /[apply].
    move=> [] ? [] H''N ?. subst.
    econstructor; [|by eassumption].
    move: H''M. congr stlc. congr sarr; apply: ssim_inj; by eassumption.
  - move=> {}Gamma' [?|s t] {}M u' {}A' /ssimE; first done.
    move=> [?] [?] [?] [[]] ?? []. subst.
    move=> ??? IH Gamma0 /= H' H'M.
    exists (sarr s t). split; [|by constructor].
    constructor. case: (IH (s :: Gamma0)).
    + by move=> [|x]; [|apply: H'].
    + by apply: allfv_impl H'M => - [|?] /=.
    + move=> ? [+] ?. congr stlc. apply: ssim_inj; by eassumption.
Qed.

Lemma ssim_arr_transport' s t u A t' A' : ssim (sarr s t) (niarr u A) -> ssim t' A' -> ssim (sarr s t') (
    niarr u A').
Proof.
  move=> /ssimE [?] [?] [?] [[-> ?]] [??] ?. by constructor.
Qed.

Lemma stlc_nitlc_ssim Gamma Gamma' M t A :
  stlc Gamma M t -> nitlc Gamma' M A -> env_ssim Gamma Gamma' -> ssim t A.
Proof.
  move=> /[dup] /stlc_allfv_not_None /nitlc_stlc + /stlc_inj HM.
  by move=> /[apply] /[apply] - [?] [/HM <-].
Qed.
```

```
Lemma nitlc_allfv_not_nil Gamma M A :
  nitlc Gamma M A ->
  allfv (fun x => nth x Gamma nil <> nil) M.
Proof.
  elim /nitlc_ind'.
  - move=> {}Gamma t x {}A ? /=. by case: (nth x Gamma []).
  - move=> {}Gamma [|Delta ?]; first done.
    move=> Gamma' ??? [|??] > _ ? H ? HM ? /=.
    + by move=> /Forall2_length.
    + move=> /Forall2E [] HN ?. split.
      * apply: allfv_impl HM => x.
        move: H => /sty_permutation_In_nth => /(_ Gamma' x).
        case: (nth x Gamma []); last done.
        move: (nth x Gamma' []) => [|B ?]; first done.
        move=> H _ _. apply: (H B); by left.
      * apply: allfv_impl HN => x.
        move: H => /sty_permutation_In_nth => /(_ Delta x).
        case: (nth x Gamma []); last done.
        move: (nth x Delta []) => [|B ?]; first done.
        move=> H _ _. apply: (H B); [by left|].
        right. by left.
  - move=> > ?? /=. by apply: allfv_impl=> - [|?] /=.
Qed.

Lemma env_ssim_merge Gamma Delta1 Delta2 :
  env_ssim Gamma Delta1 ->
  env_ssim Gamma Delta2 ->
  env_ssim Gamma (merge Delta1 Delta2).
Proof.
  move=> H1 H2 x. move: (H1 x) (H2 x).
  case: (nth_error Gamma x); last done.
  move=> *. rewrite nth_merge. by apply /Forall_app.
Qed.

Lemma env_ssim_fold_right_merge Gamma Gamma' Deltas :
  env_ssim Gamma Gamma' ->
  Forall (fun Delta => env_ssim Gamma Delta) Deltas ->
  env_ssim Gamma (fold_right merge Gamma' Deltas).
Proof.
  move=> H. elim; first done.
  move=> *. by apply: env_ssim_merge.
Qed.

Lemma env_ssim_sty_permutation_In Gamma0 Gamma Delta Deltas :
  env_ssim Gamma0 Gamma ->
  sty_permutation [Gamma] Deltas ->
  In Delta Deltas ->
  env_ssim Gamma0 Delta.
Proof.
  move=> HG0G /sty_permutation_In_nth /[apply] HDDs x.
  have := HG0G x.
  case: (nth_error Gamma0 x); last done.
  move=> s /Forall_forall H.
  apply /Forall_forall => A /HDDs. by apply: H.
Qed.

Lemma nitlc_change_K u v Gamma M A :
  nitlc (u :: Gamma) (ren S M) A ->
  nitlc (v :: Gamma) (ren S M) A.
Proof.
  apply: nitlc_allfv. apply: allfv_ren. by apply: allfv_trivial.
Qed.

Lemma ssim_arr_transport u u' A s t :
  ssim (sarr s t) (niarr u A) ->
  length u' = length u ->
  Forall (ssim s) u' ->
  ssim (sarr s t) (niarr u' A).
Proof.
  move=> /ssimE [?] [?] [?] [[??]] [??]. subst.
  case: u'; first done.
  move=> > ??. by constructor.
Qed.

Lemma hf_change_ty A B M Gamma0 Gamma :
  hf M ->
  env_ssim Gamma0 Gamma ->
  nitlc Gamma M A ->
  (forall s, ssim s A -> ssim s B) ->
  exists Gamma', env_ssim Gamma0 Gamma' /\ nitlc Gamma' M B.
Proof.
  move=> H HG0G. elim: H A B Gamma HG0G.
  - move=> x A B Gamma HG0G /nitlcE [[t']] H Hx HAB.
    exists (merge ((repeat [] x) ++ [[B]]) Gamma). split.
    + apply: env_ssim_merge; last done.
      move=> y.
      have [<-|] := Nat.eq_dec x y.
```

```
      * have := HG0G x. move: (nth x Gamma []) Hx.
        move=> ? /in_split [?] [?] ->.
        move: (nth_error Gamma0 x) => [t|]; last done.
        move=> /Forall_app [_] /Forall_cons_iff [+ _].
        move=> /HAB ?.
        rewrite app_nth2 repeat_length; first done.
        rewrite Nat.sub_diag. by constructor.
      * move: (nth_error Gamma0 y) => [?|]; last done.
        elim: (x) y; first by move=> [|[|y]].
        move=> z IH [|y]; first done.
        move=> ?. apply: IH. lia.
    + apply: nitlc_var.
      * apply: HAB. by eassumption.
      * apply: In_nth_merge_l.
        rewrite app_nth2 repeat_length; first done.
        rewrite Nat.sub_diag. by left.
  - move=> {}M N ? IH A B Gamma HG0G.
    move=> /nitlcE [Gamma'] [Deltas] [u] [HG].
    move=> [/IH {}IH] HDs HAB.
    have /IH {}IH : env_ssim Gamma0 Gamma'.
    { move: HG => /env_ssim_sty_permutation_In. by apply; [|left]. }
    have /IH [Gamma'' [??]] : forall s, ssim s (niarr u A) -> ssim s (niarr u B).
    { move=> [?|??] /ssimE; first done.
      move=> [?] [?] [?] [[??]] [??]. subst.
      constructor; first done.
      by apply: HAB. }
    exists (fold_right merge Gamma'' Deltas). split.
    + apply: env_ssim_fold_right_merge; first done.
      apply /Forall_forall => Delta ?.
      move: HG => /env_ssim_sty_permutation_In. by apply; [|right].
    + econstructor.
      * by apply: sty_permutation_fold_right_merge.
      * by eassumption.
      * done.
Qed.

(* subject expansion for specialized K-reduction *)
Theorem stepK_expansion M N Gamma0 t0 Gamma A : stepK M N -> stlc Gamma0 M t0 -> nitlc Gamma N A ->
  (* strengthen this to all variables, but not length-fixed? *)
  env_ssim Gamma0 Gamma ->
  exists Gamma',
    env_ssim Gamma0 Gamma' /\
    match M with
    | var _ =>  nitlc Gamma' M A
    | app _ _ => nitlc Gamma' M A
    | lam _ _ => exists A', nitlc Gamma' M A'
    end.
Proof.
  have Hty : forall Gamma M A,
    match M with
    | var _ => nitlc Gamma M A
    | app _ _ => nitlc Gamma M A
    | lam _ _ => exists A', nitlc Gamma M A'
    end -> exists A', nitlc Gamma M A'.
  { move=> ? [] > ? => [| |[? ?]]; eexists; by eassumption. }
  move=> H. elim: H Gamma0 t0 Gamma A.
  - (* stepKRed ... stepK (app (lam (sarr s t) (ren S M)) N) M *)
    move=> t {}M {}N /stlc_nitlc_nf H1N Gamma0 t0 Gamma A.
    move=> /stlcE [] s0 + /H1N {H1N} [GammaN] [A'] [?] [HG0N ?] H'1M.
    move=> /stlcE [] s1 t1 H1M [??] ?.
    subst=> /= HG0.
    exists (merge Gamma GammaN).
    split.
    { apply: env_ssim_merge; first done.
      elim: HG0N; first by case.
      by move=> > ?? H [|?] /=; [|apply: H]. }
    have ? : ssim t1 A.
    { move: (H1M) => /stlc_nitlc_ssim. apply.
      - by apply: (nitlc_ren _ ([A']:: Gamma)) H'1M; [lia|].
      - by move=> [|?] /=; [constructor|apply: HG0]. }
    apply: (nitlc_app _ Gamma [GammaN] _ _ [A']).
    + have := (sty_permutation_mergeL Gamma GammaN nil nil [GammaN]).
      rewrite merge_nil_r. by apply=> - [|x].
    + constructor; [by do ? constructor|].
      by apply: nitlc_ren; [lia| |eassumption].
    + by constructor.
  - (* stepKLam ... stepK (lam t M) (lam t M') *)
    move=> ? {}M M' ? IH Gamma0 t0 Gamma [] > + /nitlcE.
    { by move=> _ [?] [?] []. }
    move=> /stlcE [] s' t' + ??. subst.
    move=> /[dup] H0M /IH {}IH.
    move=> [?] [?] [[??]] [/ssimE] [?] [?] [?] [[??] [??]]. subst.
    move=> /[dup] ? /IH{IH} + /= H.
    evar (T : Type). evar (H' : T). subst T. move=> /(_ H').
    [H']: { by move=> [|x] /=; [|apply: H]. }
    move=> [Gamma'] [H4M] /Hty [?] H5M.
    exists (skipn 1 Gamma'). split.
```

```
  { move=> x. have /= := (H4M (S x)).
    case: (nth_error Gamma0 x); last done.
    by move: (x) (Gamma')=> [|?] [|??]. }
  eexists (niarr (lift_sty s' :: nth 0 Gamma' []) _). constructor.
  + constructor.
    * constructor; [|by apply: (H4M 0)].
      by apply: ssim_lift_sty.
    * apply: stlc_nitlc_ssim; by eassumption.
  + have -> : (lift_sty s' :: nth 0 Gamma' []) :: skipn 1 Gamma' = merge [[lift_sty s']] Gamma'.
    { by case: (Gamma'). }
    by apply: weakening.
- (* stepKNAppR ... stepK (app M N) (app M N') *)
  move=> {}M {}N N' ?? IH Gamma0 t0 Gamma A.
  move=> /stlcE [] s H1M H1N.
  move=> /nitlcE [Gamma'] [Deltas] [u] [HG] [H2M] H2N HG0G.
  move: (H1N) (H2N) => /IH {}IH.
  have /Forall_cons_iff [HG0G' HG0Ds]: Forall (fun Delta => env_ssim Gamma0 Delta) (Gamma' :: Deltas).
  { apply /Forall_forall => Delta /sty_permutation_In_nth H.
    move: HG => /H {}H x.
    have := HG0G x.
    move: (nth_error Gamma0 x) => [?|]; last done.
    move=> /Forall_forall H'.
    apply /Forall_forall => ? /H.
    by apply: H'. }
  have: exists u' Deltas', length u' = length u /\
    Forall2 (fun B' Delta' => env_ssim Gamma0 Delta' /\ nitlc Delta' N B') u' Deltas'.
  { elim: H2N HG0Ds; first by exists nil, nil.
    move=> B Delta u' Deltas' /IH {}IH.
    move=> ? IH' /Forall_cons_iff [/IH].
    move=> [Delta'] [? /Hty [B' ?]] /IH' [u''] [Deltas''] [??].
    exists (B' :: u''), (Delta' :: Deltas'') => /=.
    split; [by lia|by constructor]. }
  move => [u'] [Deltas'] [Hu'u HDs'].
  have [Gamma'' [??]]: exists Gamma'', env_ssim Gamma0 Gamma'' /\ nitlc Gamma'' M (niarr u' A).
  { have H' : ssim (sarr s t0) (niarr u A).
    { apply: stlc_nitlc_ssim; by eassumption. }
    have ? : ssim (sarr s t0) (niarr u' A).
    { apply: ssim_arr_transport; [by eassumption..|].
      elim: HDs'; first done.
      move=> > [HG0D ?] ??. constructor; last done.
      apply: stlc_nitlc_ssim; by eassumption. }
    apply: (hf_change_ty); [by eassumption..|].
    move=> ?. by move: H' => /ssim_inj /[apply] <-. }
  exists (fold_right merge Gamma'' Deltas').
  split.
  { apply: env_ssim_fold_right_merge; first done.
    elim: HDs'; first done.
    move=> > [??] *. by constructor. }
  apply: (nitlc_app _ Gamma'' Deltas' _ _ u').
  + by apply: sty_permutation_fold_right_merge.
  + done.
  + elim: HDs'; first done.
    move=> > [] *. by constructor.
- (* stepKLAppR ... stepK (app (lam (sarr s t) (ren S M)) N) (app (lam (sarr s t) (ren S M)) N') *)
  move=> t {}M {}N N' ? IH Gamma0 t0 Gamma A.
  move=> /stlcE [] ? /stlcE [] ?? H1M [??] ? H1N. subst.
  move=> + HG0G.
  move=> /nitlcE [Gamma'] [Deltas] [u'] [HG].
  move=> [/nitlcE] [u] [?] [[??]] [?] H3M H3N. subst.
  move: (H1N) => /IH {}IH.
  have /Forall_cons_iff [HG0G' HG0Ds]: Forall (fun Delta => env_ssim Gamma0 Delta) (Gamma' :: Deltas).
  { apply /Forall_forall => Delta /sty_permutation_In_nth H.
    move: HG => /H {}H x.
    have := HG0G x.
    move: (nth_error Gamma0 x) => [?|]; last done.
    move=> /Forall_forall H'.
    apply /Forall_forall => ? /H.
    by apply: H'. }
  have: exists u' Deltas', length u' = length u /\
    Forall2 (fun B' Delta' => env_ssim Gamma0 Delta' /\ nitlc Delta' N B') u' Deltas'.
  { elim: H3N HG0Ds; first by exists nil, nil.
    move=> B Delta u' Deltas' /IH {}IH.
    move=> ? IH' /Forall_cons_iff [/IH].
    move=> [Delta'] [? /Hty [B' ?]] /IH' [u''] [Deltas''] [??].
    exists (B' :: u''), (Delta' :: Deltas'') => /=.
    split; [by lia|by constructor]. }
  move => [u'] [Deltas'] [Hu'u HDs']. exists (fold_right merge Gamma' Deltas').
  split.
  { apply: env_ssim_fold_right_merge; first done.
    elim: HDs'; first done.
    move=> > [??] *. by constructor. }
  apply: (nitlc_app _ Gamma' Deltas' _ _ u').
  + by apply: sty_permutation_fold_right_merge.
  + econstructor.
    * apply: ssim_arr_transport; [by eassumption..|].
      elim: HDs'; first done.
      move=> B Delta u'' Deltas'' [HG0D ?] ? ?. constructor; last done.
```

```
        apply: stlc_nitlc_ssim; by eassumption.
      * apply: nitlc_change_K. by eassumption.
    + elim: HDs'; first done.
      move=> > [] *. by constructor.
  - (* stepKAAppL ... stepK (app (app M1 M2) N) (app M' N) *)
    move=> > ? IH Gamma0 t0 Gamma A.
    move=> /stlcE [] ? H1M H1N.
    move=> /nitlcE [Gamma'] [Deltas] [?] [HGamma] [H2M H2N] HG0G. subst.
    move: (H1M) (H2M) => /IH /[apply].
    case.
    { move=> x. have := HG0G x.
      move E: (nth_error Gamma0 x) => [?|]; last done.
      move: HGamma => /sty_permutation_In_nth H /Forall_forall H'.
      apply /Forall_forall => ? /H {}H.
      apply: H'. apply: H. by left. }
    move=> Gamma'' [H4M ?].
    exists (fold_right merge Gamma'' Deltas).
    have H'Gamma : sty_permutation [Gamma] [fold_right merge Gamma' Deltas].
    { apply: sty_permutation_trans; [by eassumption|].
      apply: sty_permutation_sym. by apply: sty_permutation_fold_right_merge. }
    split.
    { apply: env_ssim_fold_right_merge; first done.
      apply /Forall_forall => Delta ?.
      move: HGamma => /sty_permutation_In_nth => /(_ Delta) H x.
      have := HG0G x.
      move E: (nth_error Gamma0 x) => [?|]; last done.
      move=> /Forall_forall H'.
      apply /Forall_forall => ? /H {}H.
      apply: H'. apply: H. by right. }
    econstructor; [by apply: sty_permutation_fold_right_merge| |]; by eassumption.
Qed.

Lemma nitlc_ren_swap u0 u1 Gamma M A :
  nitlc (u1 :: u0 :: Gamma) (ren swap M) A ->
  nitlc (u0 :: u1 :: Gamma) M A.
Proof.
  move=> /(@nitlc_ren _ _ swap).
  rewrite ren_ren.
  have -> : ren (funcomp swap swap) M = M.
  { rewrite -[RHS]ren_id. by apply: ren_ext=> - [||[|?]]. }
  apply.
  - move=> [||[|?]] [||[|?]] /=; lia.
  - by move=> [||[|?]] /=.
Qed.

Lemma stepGRed_expansion (s1 s2 t : sty) (M N : tm) (Gamma : environment) (A : nity) :
  nitlc Gamma (lam (sarr s2 t) (app (lam (sarr s1 t) (ren swap M)) (ren S N))) A ->
  nitlc Gamma (app (lam (sarr s1 (sarr s2 t)) (lam (sarr s2 t) M)) N) A.
Proof.
  move=> /nitlcE [u2] [B] [->] [?].
  move=> /nitlcE [Gamma'] [Deltas] [u1] [HG] [].
  move=> /nitlcE [u1'] [A'] [[??]] [?] HM HN. subst u1' A'.
  have ? : ssim (sarr s1 (sarr s2 t)) (niarr u1 (niarr u2 B)).
  { by apply: ssim_arr_transport'; [by eassumption|]. }
  apply: (nitlc_app _ (skipn 1 Gamma') (map (skipn 1) Deltas) _ _ u1).
  + move: HG => /(sty_permutation_skipn 1). by apply.
  + econstructor; first done.
    econstructor; first done.
    apply: nitlc_ren_swap.
    have : exists u2', sty_permutation [merge ([[]; u2']) (u1 :: Gamma')] [u1 :: u2 :: skipn 1 Gamma'].
    { move: Gamma' HG {HM} => [|u Gamma'] HG.
      { by exists u2. }
      suff : exists u' : list nity, sty_permutation [[u' ++ u]] [[u2]].
      { move=> [u2'] H. by exists u2'=> - [|[|x]]; [|apply: (H 0)|]. }
      exists (concat (concat (map (firstn 1) Deltas))).
      move=> [|x]; last done.
      have /= /Permutation_sym := HG 0.
      apply: Permutation_trans.
      rewrite app_nil_r. apply: Permutation_trans; [by apply: Permutation_app_comm|].
      apply: Permutation_app; first done.
      apply: Permutation_eq.
      elim: (Deltas); first done.
      by move=> [|? Delta'] Deltas' /= <-. }
    move=> [?] /permutation. apply.
    by apply: weakening.
  + elim: HN; first done.
    move=> ? Delta > HN *. constructor; last done.
    move: HN.
    move=> /(nitlc_allfv_ren Nat.pred). rewrite ren_ren ren_id. apply.
    * apply: allfv_ren. apply: allfv_trivial => ? /=.
      apply: allfv_ren. by apply: allfv_trivial => ? /= ->.
    * apply: allfv_ren. apply: allfv_trivial => x /=.
      by move: (x) (Delta) => [|?] [|??].
Qed.

(* subject expansion for gamma-reduction *)
Theorem stepG_expansion M N Gamma A : stepG M N -> nitlc Gamma N A -> nitlc Gamma M A.
```

```
Proof.
  move=> H. elim: H Gamma A.
  - move=> >. by apply: stepGRed_expansion.
  - move=> > ? IH > /nitlcE [?] [?] [?] [? /IH ?]. subst.
    by econstructor.
  - move=> > ? IH > /nitlcE [?] [?] [?] [?] [/IH ?] ?.
    econstructor; by eassumption.
  - move=> > ? IH > /nitlcE [?] [?] [?] [?] [?] HN'.
    econstructor; [by eassumption..|].
    apply: Forall2_impl HN' => >. by apply: IH.
Qed.

Lemma sty_permutation_app_concat Deltas Gammas Gammas's :
  length Gammas = length Gammas's ->
  Forall2 (fun Delta '(Gamma, Gammas') => sty_permutation [Delta] (Gamma :: Gammas')) Deltas (combine Gammas
      Gammas's) ->
  sty_permutation Deltas (Gammas ++ concat Gammas's).
Proof.
  move=> H H' x.
  elim: Deltas Gammas Gammas's H H'.
  - by move=> [|??] [|??] /= ? /Forall2_length /=.
  - move=> ?? IH [|??] [|??] /= ? /[dup] /Forall2_length /= ?; [lia..|].
    move=> /Forall2E [] /(_ x) /= H /IH ->; [lia|].
    rewrite app_nil_r in H.
    apply: Permutation_trans; [by apply: Permutation_app; [eassumption|]|].
    rewrite !map_app !concat_app !app_assoc. apply: Permutation_app_tail.
    rewrite -!app_assoc. apply: Permutation_app_head.
    by apply: Permutation_app_comm.
Qed.

#[local] Arguments list_sum : simpl never.
#[local] Arguments map : simpl never.
#[local] Arguments concat : simpl never.

Lemma stepIRed_expansion (s t : sty) (M N : tm) :
  has_var_zero M ->
  forall Gamma0 (Gamma : environment) (A : nity),
  stlc Gamma0 (app (lam (sarr s t) M) N) t ->
  env_ssim Gamma0 Gamma ->
  nitlc Gamma (subst (scons N var) M) A ->
  nitlc Gamma (app (lam (sarr s t) M) N) A.
Proof.
  move=> H0M Gamma0 Gamma A /stlcE [] ?.
  move=> /stlcE [] ? t' H1M [??] [] ? H1N HG0G HNM. subst.
  suff: exists Gamma' u Deltas,
    sty_permutation [Gamma] (Gamma' :: Deltas) /\
    nitlc (u :: Gamma') M A /\
    ssim (sarr s t') (niarr u A) /\
    Forall2 (fun B Delta => nitlc Delta N B) u Deltas.
  { move=> [?] [?] [?] [?] [?] [H ?].
    move: (H) => /ssimE [?] [?] [?] [[??]] [??]. subst.
    econstructor; [|econstructor|]; by eassumption. }
  rename t' into t.
  have [Gamma' [u [Deltas]]] : exists (Gamma' : environment) (u : list nity) (Deltas : list environment),
    sty_permutation [Gamma] (Gamma' :: Deltas) /\
    nitlc (u :: Gamma') M A /\
    Forall (ssim s) u /\
    Forall2 (fun (B : nity) (Delta : environment) => nitlc Delta N B) u Deltas.
  { move: H1N H1M HG0G HNM. clear.
    elim /(Nat.measure_induction _ size_tm): M N t A Gamma0 Gamma. case.
    - move=> [|x] _ N t A Gamma0 Gamma /=.
      + move=> HN ? HG0G ?.
        have ? : ssim s A.
        { apply: (stlc_nitlc_ssim Gamma0 Gamma); by eassumption. }
        exists nil, [A], [Gamma]. do ? split.
        * by case.
        * by econstructor; [eassumption|left].
        * by constructor.
        * by constructor.
      + move=> H1N H1M HG0G /nitlcE [[?]] ??.
        exists Gamma, nil, []. do ? split; [done| |done..].
        econstructor; by eassumption.
    - move=> M1 M2 IH N t0 A Gamma0 Gamma H1N.
      have /IH IH1 : size_tm M1 < size_tm (app M1 M2).
      { move=> /=. lia. }
      have /IH IH2 : size_tm M2 < size_tm (app M1 M2).
      { move=> /=. lia. }
      clear IH.
      move=> /stlcE [] ? H1M1 H1M2 HG0G /=.
      move=> /nitlcE [Gamma'] [Deltas] [u] [HGDs] [H2M1] H2M2.
      have : env_ssim Gamma0 Gamma'.
      { by apply: env_ssim_sty_permutation_In; [eassumption..|left]. }
      move: (H2M1) (H1M1) (H1N) => /IH1 /[apply] /[apply] /[apply].
      move=> [Gamma''] [u''] [Deltas''].
      move=> [?] [?] [??].
      move: (H1M2) (H1N) => /IH2 /[apply] {}IH2.
      have HDs : Forall (fun Delta => env_ssim Gamma0 Delta) Deltas.
```

```
 { apply /Forall_forall => Delta ?.
   apply: env_ssim_sty_permutation_In; [by eassumption..|by right]. }
 have : Forall2 (fun B Delta => exists Gamma'' u'' Deltas'',
   sty_permutation [Delta] (Gamma'' :: Deltas'') /\
   nitlc (u'' :: Gamma'') M2 B /\
   Forall (ssim s) u'' /\
   Forall2 (fun B' Delta' => nitlc Delta' N B') u'' Deltas'') u Deltas.
 { elim: H2M2 HDs; first done.
   move=> > /IH2 {}IH2 ? IH' /Forall_cons_iff [/IH2 ?] /IH' ?.
   by constructor. }
 move=> /[dup] /Forall2_length ?.
 move=> /Forall2_exists_exists_Forall2 [Gamma's].
 move=> /[dup] /Forall2_length. rewrite combine_length=> ?.
 move=> /Forall2_exists_exists_Forall2 [us].
 move=> /[dup] /Forall2_length. rewrite !combine_length=> ?.
 move=> /Forall2_exists_exists_Forall2 [Deltass].
 move=> /Forall2_Forall [/Forall_forall H].
 rewrite !combine_length=> ?.
 exists (fold_right merge Gamma'' Gamma's).
 exists (u'' ++ concat us).
 exists (Deltas'' ++ concat Deltass). do ? split.
 + apply: sty_permutation_trans; first by eassumption.
   have : sty_permutation ([Gamma'] ++ Deltas) ((Gamma'' :: Deltas'') ++ Deltas).
   { by apply: sty_permutation_app. }
   move=> /sty_permutation_trans. apply.
   apply: (sty_permutation_trans _ (([Gamma''] ++ Deltas'' ++ Gamma's) ++ concat Deltass)).
   * rewrite -!app_assoc /=.
     apply: (sty_permutation_app (_ :: _) _ (_ :: _) _); first done.
     apply: sty_permutation_app_concat; first by lia.
     apply /Forall2_Forall. split; last by (rewrite combine_length; lia).
     apply /Forall_forall => - [Delta''' [Gamma'''' Deltas''']] H''' /=.
     have [? [?]] : exists u''' A''', In (Deltas''', (u''', (Gamma'''', (A''', Delta''')))) (combine
Deltass (combine us (combine Gamma's (combine u Deltas)))).
     { move: H''' => /In_nth_error [n].
       move=> /nth_error_combine [?].
       move=> /nth_error_combine [??].
       case E1: (nth_error us n)=> [u'''|]; first last.
       { exfalso. apply: (nth_error_length_neq Deltas us n); [congruence..|lia]. }
       case E2: (nth_error u n)=> [A'''|]; first last.
       { exfalso. apply: (nth_error_length_neq Deltas u n); [congruence..|lia]. }
       exists u''', A'''.
       apply: (nth_error_In _ n).
       by do 4 (apply /nth_error_combine; split; first done). }
     move=> /H /=. tauto.
   * apply: (sty_permutation_app _ _ (_ :: _) _); last done.
     rewrite app_assoc.
     apply: sty_permutation_trans; first by apply: sty_permutation_app_comm.
     rewrite app_assoc. apply: (sty_permutation_app _ _ [_] _); last done.
     apply: sty_permutation_sym. apply: sty_permutation_trans; first by apply:
sty_permutation_fold_right_merge.
     by apply: sty_permutation_trans; last by apply: sty_permutation_app_comm.
 + apply: (nitlc_app _ (u'' :: Gamma'') (map (fun '(u, Gamma') => cons u Gamma') (combine us Gamma's)))
;
     [|by eassumption|].
   * have : length us = length Gamma's by lia.
     elim: (us) (Gamma's).
     { move=> [|??] /= ?; last done.
       by rewrite app_nil_r=> - [|?]. }
     move=> u''' ? IH [|Gamma''' ?] /=; first done.
     move=> [/IH] {}IH [|y].
     { have := IH 0.
       rewrite !map_cons !concat_cons /= map_nil concat_nil !app_nil_r=> {}IH.
       by apply: Permutation_app_middle. }
     have := IH (S y).
     rewrite !map_cons !concat_cons /= nth_merge map_nil concat_nil !app_nil_r=> {}IH.
     by apply: (Permutation_app_middle _ []).
   * apply /Forall2_Forall. split.
     ** apply /Forall_forall=> - [] /= C ? H'''.
        move: (H''') => /in_combine_r /in_map_iff [[u''' Gamma''']] [?] ?. subst.
        have [? [?]] : exists Delta''' Deltas''',
          In (Deltas''', (u''', (Gamma''', (C, Delta''')))) (combine Deltass (combine us (combine Gamma
's (combine u Deltas)))).
        { move: H''' => /In_In_combine_l => /(_ _ Deltass). case.
          { rewrite combine_length map_length combine_length. lia. }
          move=> Deltas'''.
          move=> /In_In_combine_l => /(_ _ Deltas). case.
          { rewrite !combine_length map_length combine_length. lia. }
          move=> Delta''' H'''. exists Delta''', Deltas'''.
          move: H''' => /In_nth_error [n].
          move=> /nth_error_combine [+ ?].
          move=> /nth_error_combine [+ ?].
          move=> /nth_error_combine [?].
          rewrite nth_error_map.
          move E: (nth_error _ n)=> [[??]|]; last done.
          move: E=> /nth_error_combine [??] [??]. subst.
          apply: (nth_error_In _ n).
          by do 4 (apply /nth_error_combine; split; first done). }
```

```
          move=> /H /=. tauto.
        ** rewrite map_length combine_length. lia.
    + apply /Forall_app. split; first done.
      apply /Forall_concat.
      apply /Forall_forall => u''' .
      have : length us = length (combine Gamma's (combine u Deltas)).
      { rewrite !combine_length. lia. }
      move=> /In_In_combine_l /[apply] - [?].
      have : length Deltass = length (combine us (combine Gamma's (combine u Deltas))).
      { rewrite !combine_length. lia. }
      move=> /In_In_combine_r /[apply] - [?] /H /=. tauto.
    + apply: Forall2_app; first done.
      apply: Forall2_concat.
      apply /Forall2_Forall. split; [|lia].
      apply /Forall_forall=> - [] u''' Deltas''' H'''.
      suff [? /H /=] : exists T, In (Deltas''', (u''', T)) (combine Deltass (combine us (combine Gamma's (
   combine u Deltas)))) by tauto.
      move: H''' => /In_nth_error [n].
      move=> /nth_error_combine [Husn ?].
      move E: (nth_error (combine Gamma's (combine u Deltas)) n) => [T|].
      * exists T.
        apply: (nth_error_In _ n).
        by do 2 (apply /nth_error_combine; split; first done).
      * exfalso. move: E => /nth_error_None.
        rewrite !combine_length.
        have : nth_error us n <> None by congruence.
        move=> /nth_error_Some. lia.
  - move=> t M IH N t0 A Gamma0 Gamma H1N.
    move=> /stlcE [] s' t' H1M ?? HG0G /=. subst.
    move=> /nitlcE [u'] [B'] [?] [Hssim]. subst.
    have -> : subst (scons (var 0) (fun x : nat => ren S (scons N var x))) M =
      subst (scons (ren S N) var) (ren swap M).
    { rewrite subst_ren. by apply: subst_ext=> - [|[|x]] /=. }
    pose P n := match n with 0 => False | S _ => True end.
    have : (forall x, {P x} + {~ P x}) by (subst P; case; tauto).
    have /[apply] := allfv_dec P (ren swap M). case=> H'M.
    { have /nitlc_ren /[apply] : forall x1 x2, S x1 = S x2 -> x1 = x2 by lia.
      have -> : ren S (subst (scons (ren S N) var) (ren swap M)) = ren swap M.
      { rewrite -[RHS]subst_var ren_subst.
        apply: ext_allfv_subst. apply: allfv_impl H'M.
        by subst P=> - [|x] /=. }
      move=> H2M. exists Gamma, nil, nil. do ? constructor; [done..|].
      apply: nitlc_ren_swap. by apply: H2M. }
    have /IH {}IH : size_tm (ren swap M) < size_tm (lam (sarr s' t') M).
    { move=> /=. by rewrite size_tm_ren. }
    have : stlc (s :: s' :: Gamma0) (ren swap M) t'.
    { by apply: stlc_ren H1M=> - [|[|?]] /=. }
    have : stlc (s' :: Gamma0) (ren S N) s.
    { by apply: stlc_ren; last by eassumption. }
    have ? : Forall (ssim s') u'.
    { move: Hssim => /ssimE [?] [?] [?] [[??]] []. by subst. }
    have : env_ssim (s' :: Gamma0) (u' :: Gamma).
    { by move=> [|?] /=; [|apply: HG0G]. }
    move=> /IH /[apply] /[apply] /[apply].
    move=> [Gamma'] [u] [Deltas] [HG'Ds] [H2M] [? HuDs].
    exists (skipn 1 Gamma'), u, (map (skipn 1) Deltas). do ? split.
    + by move: HG'Ds => /(sty_permutation_skipn 1).
    + econstructor; first done.
      move: (Gamma') H2M HG'Ds => [|u0 ?].
      { move=> /(weakening [[]; u']) /=.
        by move=> /nitlc_ren_swap. }
      move=> /nitlc_ren_swap /= + /(_ 0) HG'Ds.
      have [u''' Hu'''] : exists u''', sty_permutation [[u''' ++ u0]] [[u']].
      { exists (concat (map ((nth 0)^~ []) Deltas)).
        move=> [|x]; last done.
        move: HG'Ds.
        rewrite !map_cons /= !concat_cons !map_nil !concat_nil !app_nil_r.
        move=> /Permutation_sym. apply: Permutation_trans.
        by apply: Permutation_app_comm. }
      move=> /(weakening [u''']) /=.
      apply: permutation=> - [|x] /=; last done.
      by apply: (Hu''' 0).
    + done.
    + apply: Forall2_map_r.
      apply: Forall2_impl HuDs=> ? Delta.
      move=> /(nitlc_allfv_ren Nat.pred _ (skipn 1 Delta)).
      rewrite ren_ren ren_id. apply.
      * apply: allfv_ren. apply: allfv_trivial=> ? /=.
        apply: allfv_ren. by apply: allfv_trivial=> ? /= <-.
      * apply: allfv_ren. apply: allfv_trivial.
        by move: (Delta) => [|??] [|?] /=. }
move=> [?] [HM] [Hu ?].
exists Gamma', u, Deltas. do ? split; [done|done| |done].
move: HM => /nitlc_allfv_not_nil.
move: (u) Hu => [|??].
- move=> _ H. exfalso. apply: H0M.
  apply: allfv_impl H. by case.
```

```
  - move=> ??. constructor; first done.
    apply: stlc_nitlc_ssim; [|eassumption..].
    apply: stlc_subst; [eassumption|].
    by move=> [|?] ? => [[<-]|]; [|constructor].
Qed.

(* subject expansion for I-reduction *)
Theorem stepI_expansion M N Gamma0 t0 Gamma A : stepI M N -> stlc Gamma0 M t0 ->
  env_ssim Gamma0 Gamma -> nitlc Gamma N A -> nitlc Gamma M A.
Proof.
  move=> H. elim: H Gamma0 t0 Gamma A.
  - move=> [?|s t] > ? ? t0 Gamma A H *.
    + by move: H => /stlcE [] > /stlcE [].
    + have ? : t0 = t.
      { by move: H => /stlcE [] > /stlcE [] > ?? []. }
      subst t0. apply: stepIRed_expansion; by eassumption.
  - move=> > ? IH >.
    move=> /stlcE [] ????? HG0G. subst.
    move=> /nitlcE [?] [?] [?] [H /IH {}IH]. subst.
    econstructor; first done.
    apply: IH; first by eassumption.
    move=> [|y] /=; last by apply HG0G.
    move: H => /ssimE [?] [?] [?] [[??]] [?]. by subst.
  - move=> > ? IH >.
    move=> /stlcE [] ??? HG0G. subst.
    move=> /nitlcE [?] [?] [?] [?] [/IH {}IH] ?.
    econstructor.
    + by eassumption.
    + apply: IH.
      * by eassumption.
      * apply: env_ssim_sty_permutation_In; [by eassumption..|by left].
    + done.
  - move=> > ? IH >.
    move=> /stlcE [] ??? HG0G. subst.
    move=> /nitlcE [?] [Deltas] [?] [H] [?] HN'.
    econstructor; [by eassumption..|].
    move: H (HG0G) => /env_ssim_sty_permutation_In /[apply].
    elim: HN'; first done.
    move=> ?? Delta ? /IH {}IH ? IH' H'. constructor.
    + apply: IH; first by eassumption.
      apply: H'. right. by left.
    + by apply: IH' => ? /= [<-|?]; apply: H'; [left|do 2 right].
Qed.

Theorem nitlc_expansion M N Gamma0 t Gamma A :
  stlc Gamma0 M t -> step M N -> nitlc Gamma N A -> env_ssim Gamma0 Gamma -> ssim t A ->
  exists (Gamma' : environment) (A' : nity),
    nitlc Gamma' M A' /\ env_ssim Gamma0 Gamma' /\ ssim t A'.
Proof.
  move=> + H + /[dup] ?. move: H => [|[|]].
  - move=> /stepI_expansion /[apply] /[apply] /[apply] *.
    do ? econstructor; by eassumption.
  - move=> /stepG_expansion + ? => /[apply] *.
    do ? econstructor; by eassumption.
  - move=> + /[dup] HM.
    move=> /stepK_expansion /[apply] /[apply] /[apply].
    move=> [Gamma'] [HG0G']. case: M HM=> [>?|>?|> HM [A']] *; [by eauto..|].
    exists Gamma', A'.
    split; [done|split;[done|]].
    apply: stlc_nitlc_ssim; by eassumption.
Qed.

Theorem nitlc_steps_type_inference M N Gamma0 t : stlc Gamma0 M t -> steps M N -> nf N ->
  exists Gamma A, nitlc Gamma M A /\ env_ssim Gamma0 Gamma /\ ssim t A.
Proof.
  move=> + /clos_rt_rt1n_iff H.
  elim: H.
  - move=> {}N /stlc_nitlc_nf /[apply] - [Gamma] [A] [?] [H ?].
    exists Gamma, A. do ? split; [done| |done].
    elim: H; first by case.
    move=> > ?? H [|?] /=; first done.
    by apply: H.
  - move=> > + _ IH.
    move=> /[dup] H + /[dup] H'.
    move=> /stlc_reduction /[apply] /IH /[apply].
    move=> [?] [?] [?] [??].
    apply: nitlc_expansion; by eassumption.
Qed.


Theorem nitlc_type_inference M Gamma0 t : stlc Gamma0 M t ->
  exists Gamma A,
    nitlc Gamma M A /\
    Forall2 (fun s u => u <> [] /\ Forall (ssim s) u) Gamma0 Gamma /\
    ssim t A.

Proof.
```

```
  move=> /[dup] HM /[dup] /nitlc_steps_type_inference H /stepIG_normalization [N] [HMN] [].
  move=> /stepK_normalization /[apply] - [N'] [HNN'] [/redex_count_zero_spec] /H {}H _.
  suff : steps M N'.
  { move=> /H [Gamma] [A] [H'M] [HG0G HtA].
    suff: exists Gamma', nitlc Gamma' M A /\ Forall2 (fun s u => Forall (ssim s) u) Gamma0 Gamma'.
    { move=> [Gamma'] [? H'].
      exists (merge (map (fun s => [lift_sty s]) Gamma0) Gamma'), A.
      do ? split; [by apply: weakening| |done].
      elim: H'; first done.
      move=> *. by do ? constructor; auto using ssim_lift_sty. }
    exists (merge (repeat (@nil nity) (length Gamma0)) (firstn (length Gamma0) Gamma)). split.
    - apply: weakening.
      apply: nitlc_allfv H'M.
      move: HM => /stlc_allfv_not_None. apply: allfv_impl.
      clear. elim: Gamma0 Gamma.
      + by move=> ? [|?].
      + move=> ?? IH [|??]; first done.
        move=> [|?]; first done.
        move=> /IH. by apply.
    - elim: Gamma0 Gamma HG0G {H'M HM H}.
      + by move=> [|??] /=.
      + move=> ? Gamma0 IH [|??] /=.
        * move=> ?. constructor; first done.
          clear. elim: Gamma0; first done.
          move=> *. by constructor.
        * move=> HG0G. constructor; first by apply: (HG0G 0).
          apply: IH=> x. by apply: (HG0G (S x)). }
  move: HMN HNN'=> /clos_rt_rt1n_iff. elim.
  - move=> ? /clos_rt_rt1n_iff. elim.
    + move=> ?. apply: rt_refl.
    + move=> > *. apply: rt_trans; [|by eassumption].
      apply: rt_step. right. by right.
  - move=> M1 M2 M3 H' /clos_rt_rt1n_iff HM2M3 _ HM3N'.
    apply: (@rt_trans _ _ _ M2).
    + apply: rt_step. rewrite /step. tauto.
    + apply: (@rt_trans _ _ _ M3).
      * rewrite /step. elim: HM2M3; [|by eauto using clos_refl_trans..].
        move=> *. apply: rt_step. tauto.
      * rewrite /step. elim: HM3N'; [|by eauto using clos_refl_trans..].
        move=> *. apply: rt_step. tauto.
Qed.


Check nitlc_type_inference.

Print Assumptions nitlc_type_inference.
```